

Technical Report

The ALU Logo dApp: From Smart Contracts to a Web Application

Individual Assignment 2: DApp Integration with Smart Contract (the ALU official logo)

Cryptocurrency and Digital Assets

Student: Joseph Nishimwe

Date: 11 November 2026

GitHub repository:
https://github.com/josep-prog/Individual_Assignment2-DApp-Integration_smartContract

Demo video: https://youtu.be/fiC_qeBMTC0

Abstract

ALU's logo is just an image file, so anyone can copy it, change it and pass it off as the real one. This project records the logo's fingerprint on the Ethereum blockchain, splits its ownership into one million shares, and adds a web application (a dApp) that connects to those contracts. Anyone who opens the web application, which currently runs on a local Hardhat blockchain, can upload a logo and find out at once whether it is genuine without writing any code, while ALU can register logos and distribute shares through a Web3 wallet.

Introduction

A logo is a university's signature. ALU's logo appears on certificates, transcripts, websites, merchandise and social media. Because it is just an image file, anyone can download it, change a colour or a letter, and put it on a fake certificate or a fake ALU page. Today there is no quick, public way for someone to check whether the logo in front of them is the real one. ALU's mission of developing ethical, entrepreneurial African leaders depends on employers and partners being able to trust its name and its credentials, and a logo that can be forged puts that trust at risk.

Why not simply keep the official copy on an ALU computer? Whoever runs that computer can change or delete the record without anyone noticing, everyone else has to take it on trust, and if it goes offline nobody can check anything. A trademark protects the design in law, but it is slow and cannot tell anyone in seconds whether a particular file is the exact original. A blockchain fills that gap. It is a public record book copied across thousands of computers around the world, where every entry is dated, anyone can check it for free, and nothing can be quietly changed afterwards. It does not replace the law; it adds proof that supports it.

The solution has two parts. The first is a digital certificate for the logo, kept on the Ethereum blockchain, that records the logo's fingerprint, who registered it and when. Anyone can ask it: "Is this file the real logo?" The second turns ownership of the logo into one million equal shares, called ALU Logo Tokens (ALUT), that ALU can give to staff, students and partners. The certificate proves which logo is the real one; the shares record who owns how much of it.

The report first sets out the plan for protecting the logo and follows each piece as it was built, tested and put live. The final part covers the gap the contracts alone could not fill: they have no screen, no file picker and no way to explain their answer, so only someone able to run Hardhat commands could use them. The web application built on top of them lets anyone check a logo in a browser, and lets ALU register logos and hand out shares through a wallet.



The ALU logo file (alu-logo.png, 1024 × 384 pixels) registered in this project.

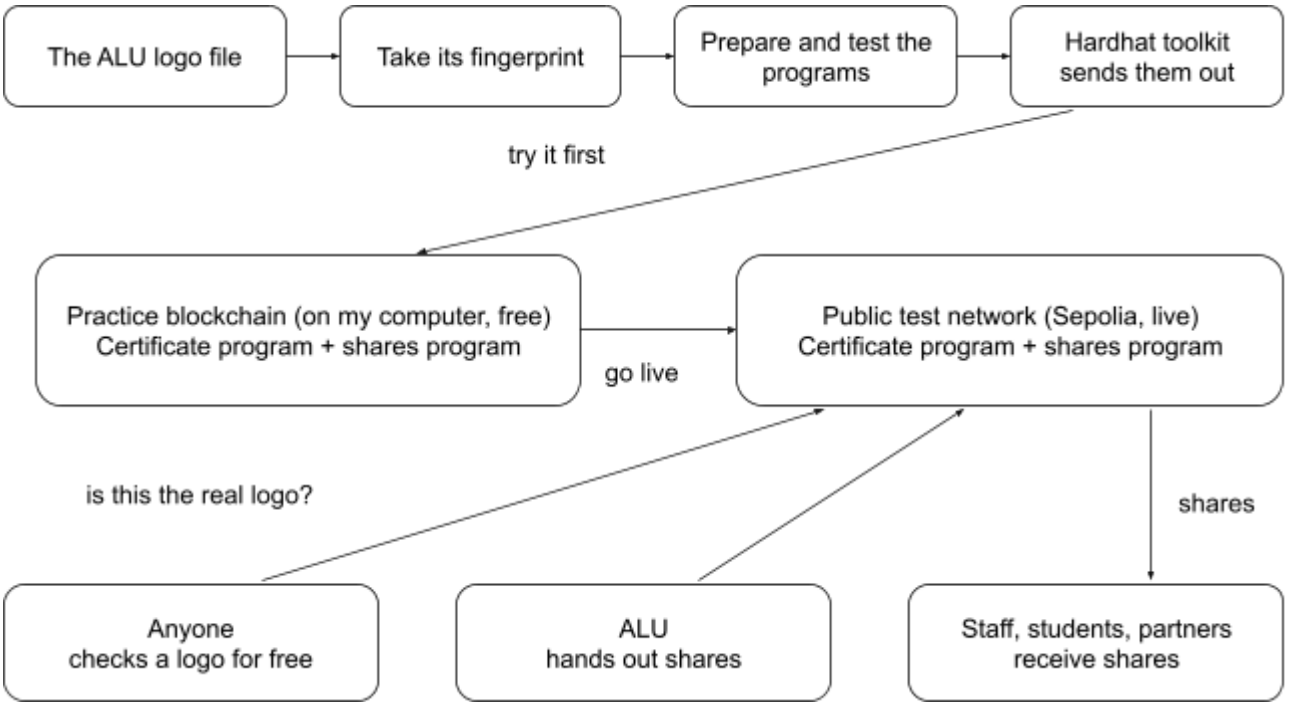
Protecting the ALU logo

The sections from here up to “Web Application: Architecture Overview” are background from Formative 1, describing the contracts this dApp reuses. The plan has four steps. First, set up a private practice blockchain on my own computer, where mistakes cost nothing. Second, take the logo’s fingerprint and store it in a digital certificate on the blockchain, so anyone can check a logo file against it. Third, create one million ownership shares in the logo that ALU can hand out. Finally, put everything live on Ethereum’s public test network and prove it works with automated tests. The next section shows how all the pieces fit together, and the sections after it follow the four steps in order.

How the pieces fit together

The big picture

Three choices shaped the design. Only the logo’s fingerprint is stored on the blockchain, not the image itself, which keeps it cheap while still revealing any change to the file. Proving the logo is genuine and sharing its ownership are separate jobs, so each has its own small program. And both programs are built from trusted, ready-made OpenZeppelin parts. The diagram shows how everything connects. On my computer, the logo is fingerprinted and the programs are prepared and tested. They are then placed on a blockchain: first the practice one, then Ethereum’s public test network. Finally, people use them: anyone can check a logo for free, ALU hands out shares, and staff, students and partners receive them.



How the system fits together: my computer, the two blockchains, and the people who use it.

What each program keeps

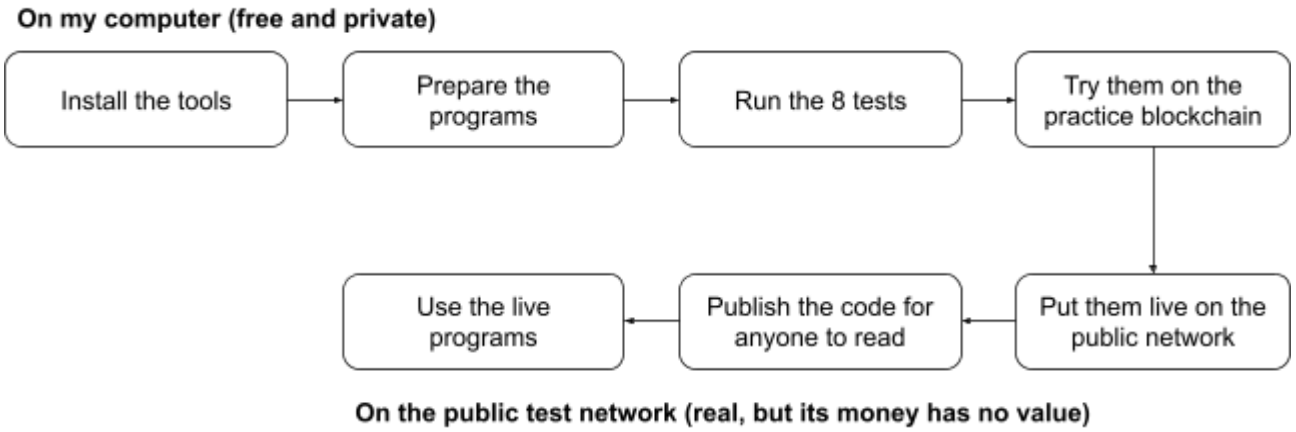
Each program keeps its own records. The certificate program, ALUAssetRegistry, keeps a registration form for each logo (its name, file type, fingerprint, who registered it and when) and a list of fingerprints already registered, so the same file can never be registered twice. The shares program, ALULogoToken, keeps the token's name and symbol, the fixed total of one million shares, how many each account holds, and which account is the owner allowed to hand them out.

The certificate program **one-of-a-kind certificate for the logo**	**The shares program** **1,000,000 identical shares in the logo**
Registration form, one per logo name, type of file, fingerprint, who registered it and when	Who holds what how many shares each account owns
List of fingerprints already registered so no file can be registered twice	The owner: ALU's account, the only one allowed to hand out shares
Lets people: register a logo, check a logo, look up its record	Lets people: hand out shares, check what percentage someone owns

What each program keeps, and what it lets people do.

From my computer to the live network

The diagram shows the order the work followed, from my computer to the public network. Nothing reached the public network until it had passed every step on my computer.



The path from my computer to the live network.

Blockchain Setup

The project is built with Hardhat, a free toolkit for Ethereum developers that can, among other things, run a complete private copy of Ethereum on one computer. The programs themselves are written in Solidity, Ethereum's programming language, and their standard building blocks come from OpenZeppelin, a widely used and security-checked library, so nothing risky had to be written from scratch.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ node -v && npm -v && npx hardhat
 --version && grep -m1 'version:' hardhat.config.js && grep -m1 '"version"' node_modules/@openzeppel
in/contracts/package.json
v22.23.3
10.9.9
2.29.1
    version: "0.8.24",
  "version": "5.0.2",
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The tools installed for the project.

The project keeps each kind of work in its own folder: the blockchain programs, the scripts that put them live, and the automated tests.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ tree -I 'node_modules|artifacts|
cache|screenshots'
.
├── alu-logo.png
├── alu-logo.svg
├── contracts
│   ├── ALUAssetRegistry.sol
│   └── ALULogoToken.sol
├── deployments
│   ├── localhost.json
│   └── sepolia.json
├── hardhat.config.js
├── package.json
├── package-lock.json
├── Project_Report.pdf
├── report
│   ├── build-report.js
│   └── Project_Report.html
├── scripts
│   ├── deploy.js
│   ├── hash-logo.js
│   └── interact.js
└── test
    └── ALUAssetRegistry.test.js

6 directories, 16 files
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The project folders.

Preparing the programs

Before a program can run on Ethereum, it is translated into the low-level instructions the network understands. Both programs, together with the OpenZeppelin parts they use, translated without errors.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ npx hardhat clean && npx hardhat
 compile
Compiled 17 Solidity files successfully (evm target: paris).
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

Both programs translated without errors.

Starting the practice blockchain

With one command, Hardhat starts the practice blockchain and creates 20 test accounts, each loaded with 10,000 play-money ETH. The secret keys for these accounts are published openly, which is why they must never be used on the real network.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ npx hardhat node | head -n 28
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (10000 ETH)
Private Key: 0x92db14e403b83dfe3df233f83dfa3a0d7096f21ca9b0d6d6b8d88b2b4ec1564e
```

The practice blockchain running, with the test accounts it created.

Why practise on a local blockchain?

A local blockchain behaves like the real Ethereum network, with accounts, balances and payments, but only my computer is connected to it. On the real network every action costs real money, takes several seconds and is permanent, so a mistake cannot be undone. The practice version is free, instant and can be wiped clean at any time. It is the training pitch before the real match.

What is a wallet address?

A wallet address, such as 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266, is a public account number on the blockchain that anyone can send to. It comes from a secret key, and whoever holds that key controls the account. On the blockchain, an address stands for an owner: in this project, the address that registers the logo is recorded as the one who registered it, and the addresses holding ALUT own those shares. Think of the address as a mailbox number that is safe to share, and the secret key as the key to the mailbox, which must never be shared.

Smart Contract Explanation

This part registers the real ALU logo on the blockchain. It needs a program that stores the logo's fingerprint and details, refuses to register the same file twice, and lets anyone check a file against the record.

What is a smart contract?

A smart contract is a program that lives on a blockchain and runs exactly as written whenever someone uses it. An ordinary program runs on one company's computer, and that company can change it, edit its data or switch it off. A smart contract runs on thousands of independent computers that check each other's work, so once it is in place nobody can secretly change it, everything it does is public, and it keeps working even if its creator disappears. An ordinary program is like a shop where the owner can change the price on your receipt; a smart contract is like a vending machine in a public square, with the rules printed on the front and everyone watching.

What is an NFT, and why use one for the logo?

An NFT, or non-fungible token, is a one-of-a-kind digital certificate. "Fungible" means interchangeable: one 1,000-franc note is as good as any other. "Non-fungible" means each item is unique, like a land title or a passport. ERC-721 is the common Ethereum standard for NFTs: each token has its own number and exactly one owner, and wallets and websites already know how to display it. There is only one official ALU logo, so it should be one unique certificate with one clear owner and its own details, which is exactly what ERC-721 provides.

The logo's fingerprint

A SHA-256 hash is a digital fingerprint of a file: a 64-character code calculated from every part of it. The same file always gives the same fingerprint, but it is practically impossible to find two different files with the same one, or to rebuild the file from its fingerprint.

I took the official ALU logo, saved it in the project as alu-logo.png, and calculated its fingerprint with a small script. A second, independent tool gave exactly the same result. The logo's fingerprint is:

838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76

This is the value stored on the blockchain when the logo is registered.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ npm run hash && sha256sum alu-lo
go.png

> assignment1-cryptocurrency_and_digital_assets@1.0.0 hash
> node scripts/hash-logo.js

File:           /home/joe/Documents/Assignment1-Cryptocurrency_and_Digital_Assets/alu-logo.png
Size (bytes):   25271
SHA-256:        838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76
bytes32 value:  0x838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76
838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76  alu-logo.png
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The logo's fingerprint, calculated by the project's script and confirmed by a second tool on the last line.

Why does changing one pixel change the whole fingerprint? SHA-256 is designed so that changing even the smallest part of a file scrambles the result completely and unpredictably. A logo with one pixel changed therefore has a fingerprint that looks nothing like the original. I tested this: adding a single invisible extra character to the end of the logo file produced an entirely different fingerprint, and the program answered "Warning: logo does not match." This is what makes the fingerprint a reliable tamper alarm.

The registration form in the program

The architecture section introduced the registration form and the list of fingerprints. In the program, the form holds five details, and a second list finds a logo's form from its certificate number. Both lists are public, so anyone can read them.

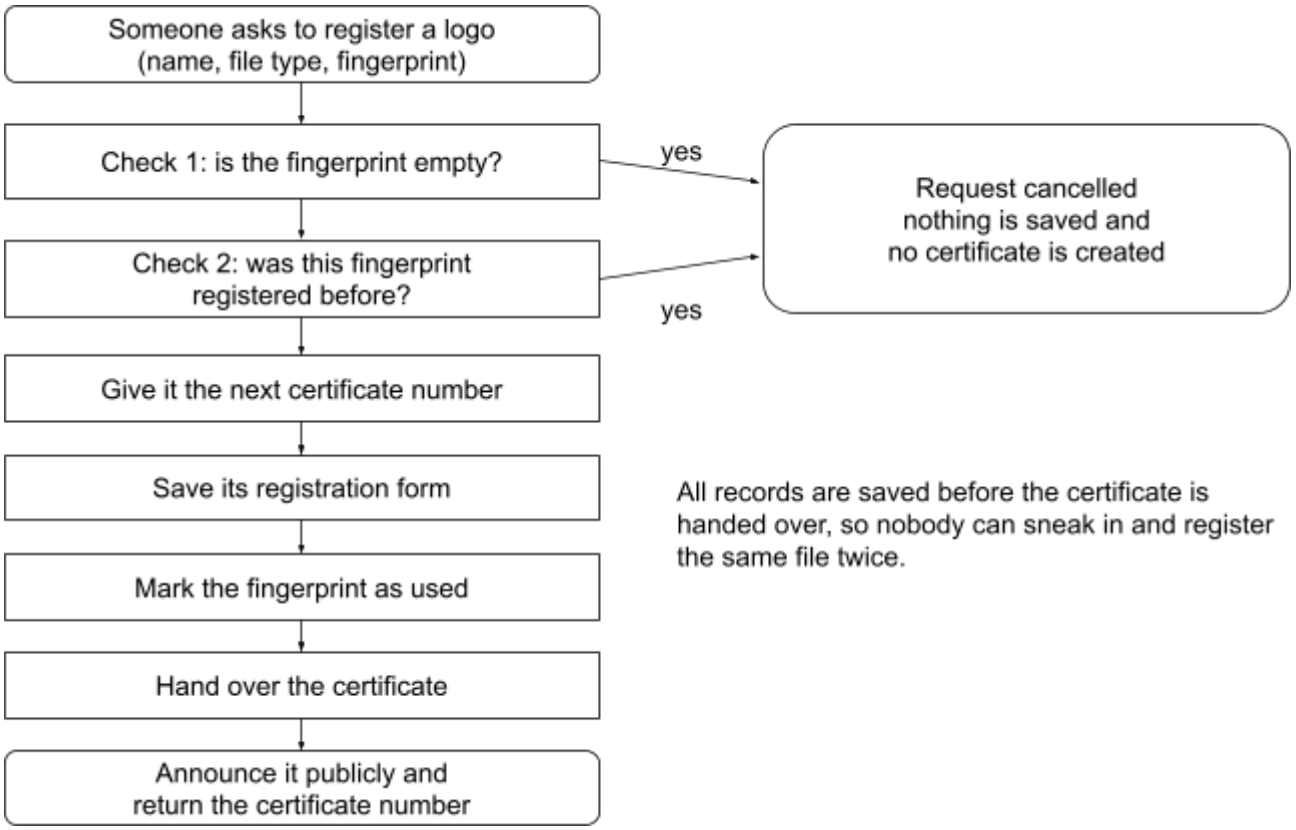
```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ pygmentize -O style=monokai -g contracts/A
LUAssetRegistry.sol | cat -n | sed -n 13,31p
    13  contract ALUAssetRegistry is ERC721 {
    14      /// @notice Registration details stored for every asset.
    15      struct AssetMetadata {
    16          string assetName; // Human-readable name, e.g. "ALU Official Logo"
    17          string fileType; // MIME type of the file, e.g. "image/png"
    18          bytes32 contentHash; // SHA-256 hash of the exact file contents
    19          address registeredBy; // Wallet that registered the asset
    20          uint256 registeredAt; // Unix timestamp of the registration block
    21      }
    22
    23      /// @notice Token ID => registration details. Public so anyone can read it.
    24      mapping(uint256 => AssetMetadata) public assets;
    25
    26      /// @notice Content hash => whether it has already been registered.
    27      /// @dev Used to reject duplicate registrations of the same file.
    28      mapping(bytes32 => bool) public registeredHashes;
    29
    30      /// @dev Number of assets registered so far. Token IDs start at 1.
    31      uint256 private tokenCounter;
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The registration form and the two lists, as written in the program.

Registering the logo

Registering is a single action. The program first runs two checks: the fingerprint must not be empty, and it must not have been registered before. If either check fails, the whole request is cancelled and nothing is saved. If both pass, the program gives the logo the next certificate number (the first is 1), fills in its form with the registering account and the current time, marks the fingerprint as used, issues the certificate to the registering account, and announces the registration publicly.

The order of these steps matters for safety. All the records are saved before the certificate is handed over, so a malicious program on the receiving end gets no chance to slip back in and register the same file twice.



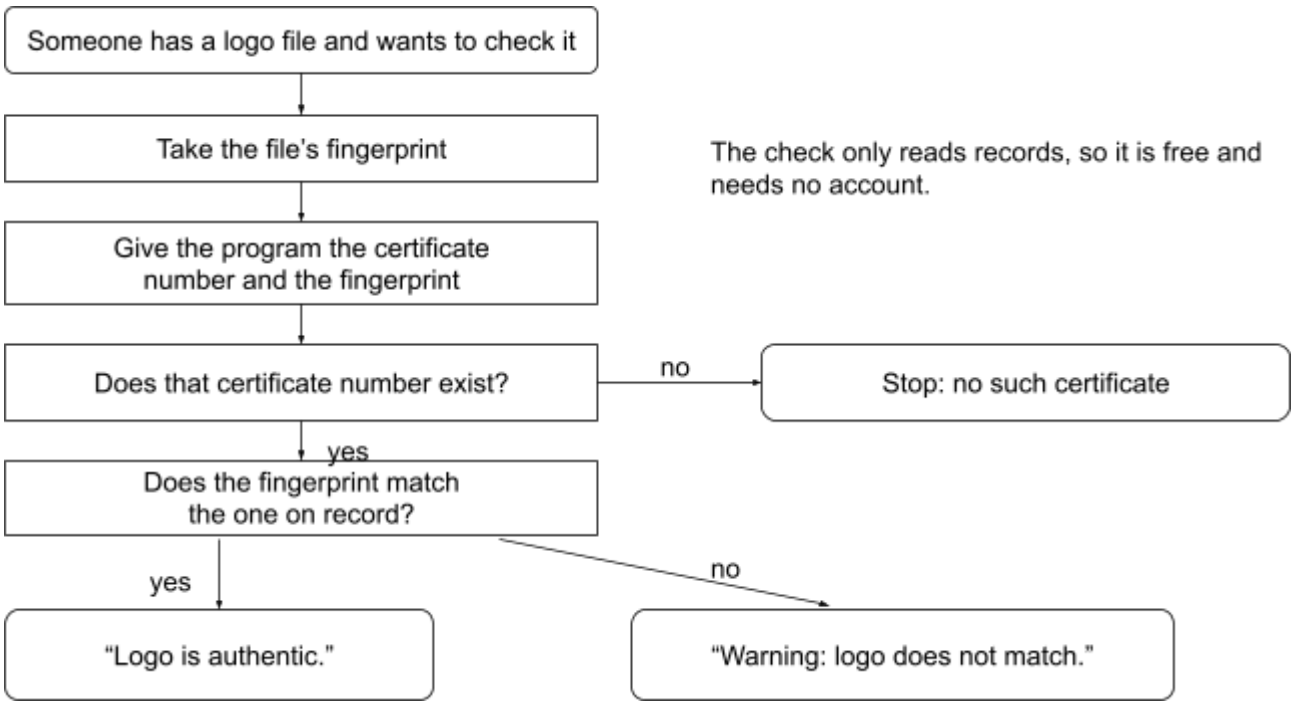
The registration process: two checks that can cancel the request, then the steps that create the certificate.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ pygmentize -O style=monokai -g contracts/A
LUAssetRegistry.sol | cat -n | sed -n 54,83p
    54      function registerAsset(
    55          string memory assetName,
    56          string memory fileType,
    57          bytes32 contentHash
    58      ) public returns (uint256) {
    59          require(contentHash != bytes32(0), "Content hash cannot be empty.");
    60          require(
    61              !registeredHashes[contentHash],
    62              "This file is already registered."
    63          );
    64
    65          tokenCounter++;
    66          uint256 tokenId = tokenCounter;
    67
    68          assets[tokenId] = AssetMetadata({
    69              assetName: assetName,
    70              fileType: fileType,
    71              contentHash: contentHash,
    72              registeredBy: msg.sender,
    73              registeredAt: block.timestamp
    74          });
    75
    76          registeredHashes[contentHash] = true;
    77
    78          _safeMint(msg.sender, tokenId);
    79
    80          emit AssetRegistered(tokenId, msg.sender, contentHash);
    81
    82          return tokenId;
    83      }
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The registration code: the two checks come first, and the records are saved before the certificate is issued.

Checking a logo

This is the authenticity checker. Someone with a logo file calculates its fingerprint and gives it to the program together with the certificate number. The program compares it with the fingerprint it stored and answers either “Logo is authentic.” or “Warning: logo does not match.” Because it only reads records and changes nothing, the check is free and does not even need an account.

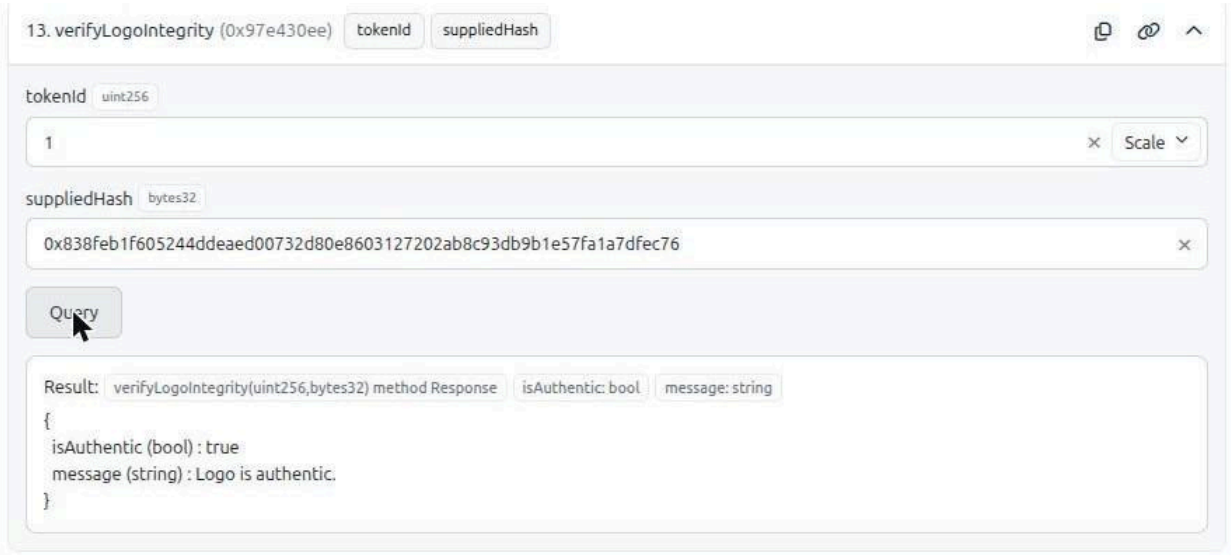


How a logo check works.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ pygmentize -O style=monokai -g contracts/A
LUAssetRegistry.sol | cat -n | sed -n 92,116p
    92      function verifyLogoIntegrity(
    93          uint256 tokenId,
    94          bytes32 suppliedHash
    95      ) public view returns (bool isAuthentic, string memory message) {
    96          _requireOwned(tokenId);
    97
    98          if (assets[tokenId].contentHash == suppliedHash) {
    99              return (true, "Logo is authentic.");
   100          }
   101
   102          return (false, "Warning: logo does not match.");
   103      }
   104
   105      /// @notice Returns the full registration details of an asset.
   106      /// @dev Reverts if the token does not exist.
   107      /// @param tokenId The NFT to look up.
   108      /// @return The stored AssetMetadata struct.
   109      function getAsset(
   110          uint256 tokenId
   111      ) public view returns (AssetMetadata memory) {
   112          _requireOwned(tokenId);
   113
   114          return assets[tokenId];
   115      }
   116
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The code that checks a logo and looks up its record.

Because the program is public, anyone can run this check from a web browser without installing anything. Here it is run from Etherscan, a public blockchain website, against the live program, using the real logo’s fingerprint:



The live check on Etherscan answers "Logo is authentic."

Looking up the record

A third action returns the full registration form for any certificate number, so anyone can see what was registered, by whom and when. Like the check, it is free. A small helper also reports how many items have been registered so far.

Putting the logo on the blockchain

One script does the whole job in a single step. It calculates the real logo's fingerprint, places the certificate program on the blockchain, registers the logo, double-checks the result, places the shares program on the blockchain, and saves the addresses where both can be found.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ npm run deploy:local

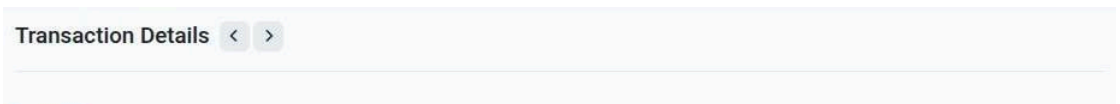
> assignment1-cryptocurrency_and_digital_assets@1.0.0 deploy:local
> hardhat run scripts/deploy.js --network localhost

Network: localhost
Deployer: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Logo SHA-256: 0x838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76

Deployment complete
Registry address: 0x5FbDB2315678afecb367f032d93F642f64180aa3
Registration tx: 0xb6303a2193d9caaa4419184ecd8167e31c358331f2c7c70882ed21bb1fb96437
Logo token ID: 1
Registered name: ALU Official Logo
Registered hash: 0x838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76
Integrity check: true - Logo is authentic.
ALUT contract address: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
ALUT total supply: 1000000
Owner percentage: 100%
Addresses saved to deployments/localhost.json
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

On the practice blockchain: both programs in place, the logo registered as certificate number 1, the check passing, and all shares held by ALU.

The same script was then run on Ethereum's public test network. Etherscan shows the real registration: its status is Success, and it created certificate number 1 for ALU's account.



The logo's registration on the public test network, as shown by Etherscan.

Why the data cannot be changed

Every block of records on the blockchain contains the fingerprint of the block before it, so changing an old record would break the chain from that point on and be spotted at once. Thousands of computers keep their own copy and check every new block, so a changed copy simply disagrees with everyone else and is rejected. Rewriting history would mean controlling most of the billions of dollars staked to run Ethereum, and the attacker would lose that stake. Finally, the certificate program has no way to edit or delete a registration, so not even ALU can change the stored fingerprint.

Tokenisation

With the logo registered, ALU wants to share its ownership. This part explains how that ownership is cut into equal shares and who can hand them out.

What is tokenisation?

Tokenisation means turning ownership of something into digital tokens. Instead of one owner holding the whole thing, ownership is cut into many equal pieces that can be held, given or passed on like money. ALU starts with all 1,000,000 ALUT, which is 100% ownership, and can then give shares to the people who build the ALU brand: the administration and board, who keep the majority so ALU stays in control; faculty, whose teaching and research give the logo its value; students and alumni, who carry the logo into the world; and partners such as an official merchandise maker, whose share could reflect a licensing deal. Later, shares could be used to vote on how the logo is used or to split any income it earns.

One certificate, many shares

	ERC-721 (NFT)	**ERC-20 (fungible token)**
Each unit is	Unique, with its own ID and data	Identical to every other unit
Divisible	No: you own token 1 or you do not	Yes: you can own 1, 500 or 100,000
Real-world example	A land title or a diploma	Money or company shares
Used in this project for	The logo itself (ALUAssetRegistry)	Ownership shares (ALULogoToken)

The logo itself is one unique item with its own fingerprint and date, so it uses the certificate standard, ERC-721. Shares must be identical, countable and divisible, so they use the standard for interchangeable tokens, ERC-20: 100,000 ALUT are worth the same whoever holds them.

Creating the shares

When the shares program is first placed on the blockchain, it names the token "ALU Logo Token" with the symbol ALUT, makes ALU's account its owner, and creates all 1,000,000 shares in that account. There is no way to create more later, so the total can never grow.

Shares also come in whole units only, so one ALUT always means exactly one share, never a

fraction.

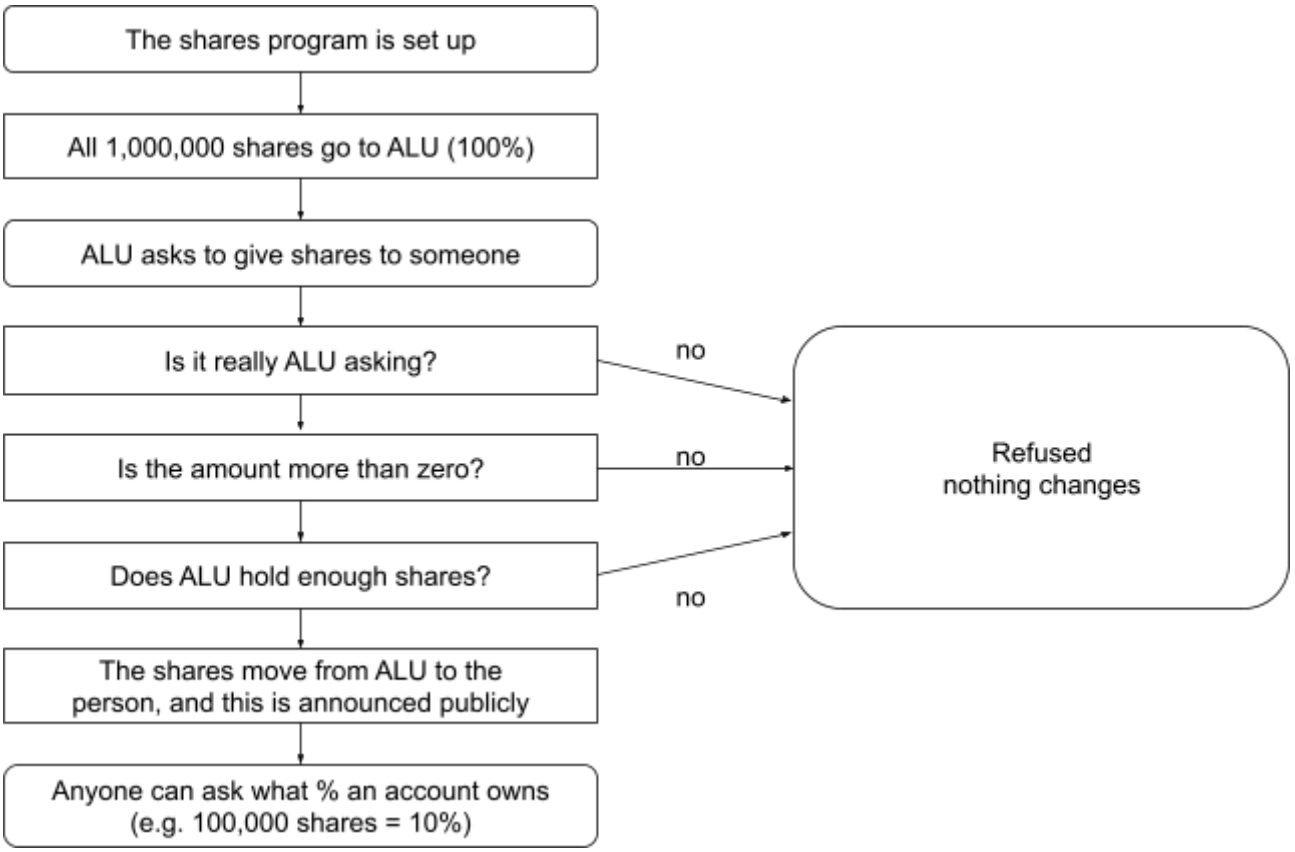
```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ pygmentize -O style=monokai -g contracts/A
LULogoToken.sol | cat -n | sed -n 14,37p
    14  contract ALULogoToken is ERC20, Ownable {
    15      /// @notice Fixed number of ownership shares.
    16      uint256 public constant TOTAL_SHARES = 1_000_000;
    17
    18      /// @notice Emitted when the owner hands out shares to a stakeholder.
    19      /// @param recipient The wallet receiving the shares.
    20      /// @param amount The number of ALUT transferred.
    21      event SharesDistributed(address indexed recipient, uint256 amount);
    22
    23      /// @notice Creates the token and gives the full supply to the logo owner.
    24      /// @param logoOwner Wallet of the original logo owner. It receives all
    25      ///        1,000,000 ALUT and becomes the contract owner.
    26      constructor(
    27          address logoOwner
    28      ) ERC20("ALU Logo Token", "ALUT") Ownable(logoOwner) {
    29          _mint(logoOwner, TOTAL_SHARES);
    30      }
    31
    32      /// @notice Shares are whole units, so the token has no decimal places.
    33      /// @dev Without this override ERC-20 defaults to 18 decimals and 1 ALUT
    34      ///      would be stored as 10^18 units.
    35      function decimals() public pure override returns (uint8) {
    36          return 0;
    37      }
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The code that creates the shares and gives them all to ALU.

Handing out shares and checking ownership

Handing out shares is locked so that only ALU's owner account can do it; anyone else is turned away. The amount must be more than zero, and ALU must hold enough shares. The shares then move from ALU to the recipient, and the transfer is announced publicly.

Anyone can also ask, for free, what percentage of the logo an account owns. The answer is a whole number: an account holding 500,000 ALUT owns 50%. Holdings below 1% show as 0.



How shares are created, handed out and measured.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ pygmentize -O style=monokai -g contracts/A
LULogoToken.sol | cat -n | sed -n 39,65p
    39      /// @notice Transfers ownership shares from the owner to a stakeholder,
    40      ///         such as faculty, students or administration.
    41      /// @dev Only the contract owner can call this. Reverts on a zero amount,
    42      ///      the zero address, or if the owner does not hold enough ALUT.
    43      /// @param recipient The wallet receiving the shares.
    44      /// @param amount The number of ALUT to transfer.
    45      function distributeShares(
    46          address recipient,
    47          uint256 amount
    48      ) public onlyOwner {
    49          require(amount > 0, "Amount must be greater than zero.");
    50
    51          _transfer(owner(), recipient, amount);
    52
    53          emit SharesDistributed(recipient, amount);
    54      }
    55
    56      /// @notice Returns a wallet's ownership share as a whole-number percentage.
    57      /// @dev Rounds down, so holdings under 1% (fewer than 10,000 ALUT) return 0.
    58      /// @param account The wallet to check.
    59      /// @return The percentage of the total supply held, from 0 to 100.
    60      function ownershipPercentage(
    61          address account
    62      ) public view returns (uint256) {
    63          return (balanceOf(account) * 100) / totalSupply();
    64      }
    65  }
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

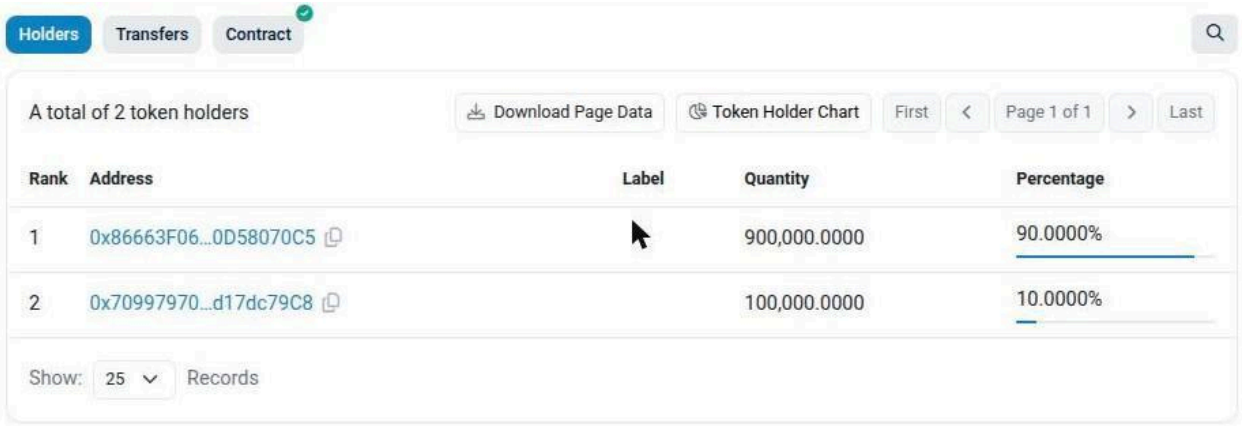
The code for handing out shares and checking ownership.

What holding 100,000 ALUT means

100,000 out of 1,000,000 is 10%, so a faculty member with 100,000 ALUT owns one tenth of the logo's shares, and anyone can confirm it. If ALU earned licensing income from the logo, they would be entitled to 10% of it; if decisions about the logo were put to a vote, they would hold 10% of the votes; and they could keep the shares or pass them on. One limit matters: the

blockchain records the share, but rights such as income only exist if ALU also writes them into a real agreement. The blockchain records ownership; the law gives it force.

I tested this on the public test network by sending 100,000 ALUT to a second account. Afterwards the program reported 90% for ALU and 10% for the other account, and Etherscan shows the same split publicly:



Share holders on Etherscan: ALU holds 900,000 ALUT (90%) and the second account 100,000 ALUT (10%).

Going live on a public network

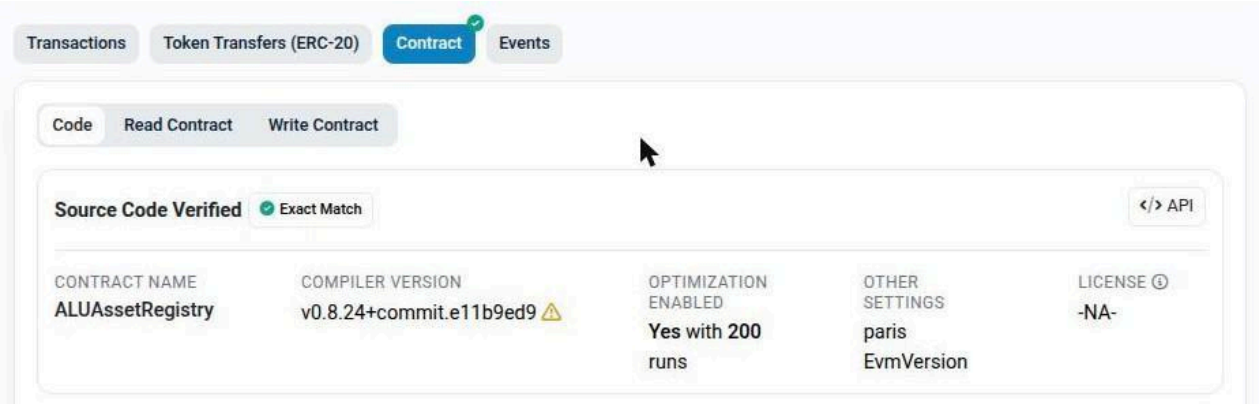
Once everything worked on my computer, both programs were placed on Sepolia, Ethereum's official public test network. Sepolia works exactly like the real Ethereum network, with real computers and public websites to inspect it, but its currency is free and has no value. Running there proves the project works on a real, shared blockchain and not only on my computer. The program code was also published on Etherscan, so anyone can read it and confirm it matches what is running.

Where to find it

Item	Value
ALUAssetRegistry (NFT)	0xA471fB76f0c2432Faf36d5f699C8868FAF0ab3B9
ALULogoToken (ALUT)	0x9C0B96aA3A3b564a03fad5b53d23775d066d44C3
Deployer and logo owner	0x86663F06852fEcCb60356841408dDcE0D58070C5
Logo registration transaction	0x2825a66b342c5d94d493e6b2b1570307ee08d38129651d039fd57155075c28cc
Share distribution transaction	0x07f305a74cd818312cc0badfdb235e0507bad889fd5076707a43ef4f1f30665e
Logo token ID	1
Registered at	27 October 2026, 09:56:36 UTC

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ cat deployments/sepolia.json
{
  "network": "sepolia",
  "chainId": 11155111,
  "deployer": "0x86663F06852fEcCb60356841408dDcE0D58070C5",
  "logoSha256": "0x838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76",
  "ALUAssetRegistry": "0xA471fB76f0c2432Faf36d5f699C8868FAF0ab3B9",
  "logoTokenId": "1",
  "registrationTx": "0x2825a66b342c5d94d493e6b2b1570307ee08d38129651d039fd57155075c28cc",
  "ALULogoToken": "0x9C0B96aA3A3b564a03fad5b53d23775d066d44C3",
  "deployedAt": "2026-10-27T09:56:51.395Z"
}joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

The record of the public test network launch, saved by the setup script.



Etherscan confirms that the published code exactly matches the program running on the network.

Trying it on the live network

A script then used every feature of the live programs. It read the logo's record, checked the real logo (authentic) and a copy with one tiny change (rejected), and read both ownership percentages, 90% and 10%.

```
Terminal
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$ npm run interact:sepolia

> assignment1-cryptocurrency_and_digital_assets@1.0.0 interact:sepolia
> hardhat run scripts/interact.js --network sepolia

Network: sepolia

getAsset:
  Name:          ALU Official Logo
  File type:     image/png
  Hash:          0x838feb1f605244ddeaed00732d80e8603127202ab8c93db9b1e57fa1a7dfec76
  Registered by: 0x86663F06852fEcCb60356841408dDcE0D58070C5
  Registered at: 2026-10-27T09:56:36.000Z

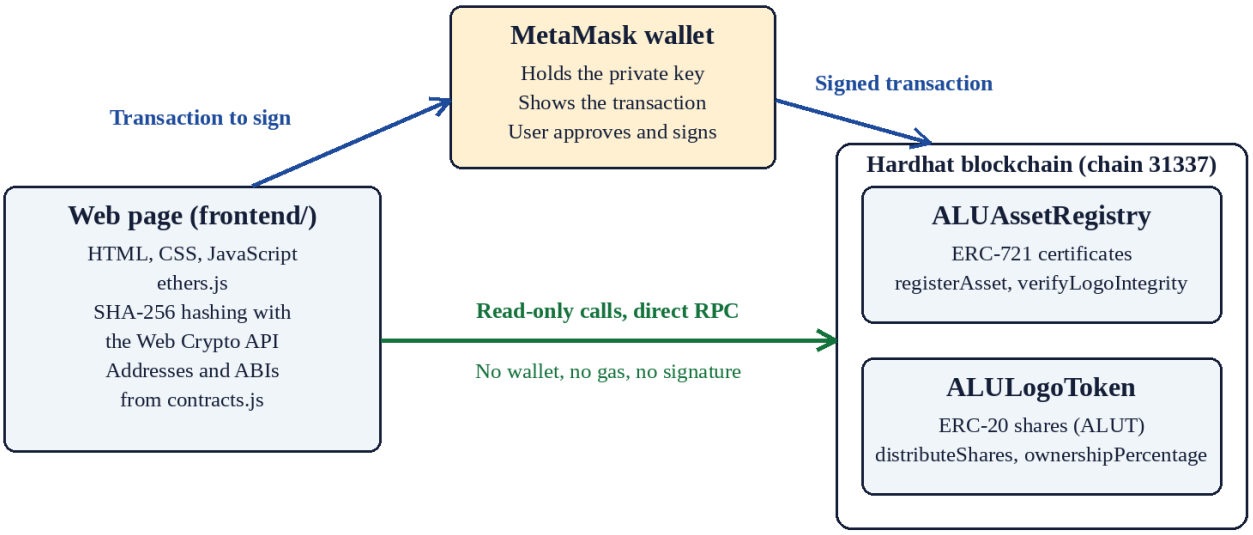
verifyLogoIntegrity (real logo):  true - Logo is authentic.
verifyLogoIntegrity (tampered):   false - Warning: logo does not match.

ownershipPercentage:
  Owner:       90% (900000 ALUT)
  Stakeholder: 10% (100000 ALUT)
joe@code:~/Documents/Assignment1-Cryptocurrency_and_Digital_Assets$
```

Using the live programs: the real logo passes, the altered copy fails, and the shares stand at 90% and 10%.

Web Application: Architecture Overview

The application has three parts: the web page, the user's wallet and the two contracts. There is no application server and no database. The page is plain HTML, CSS and JavaScript, served as static files, and the blockchain holds all of the data.

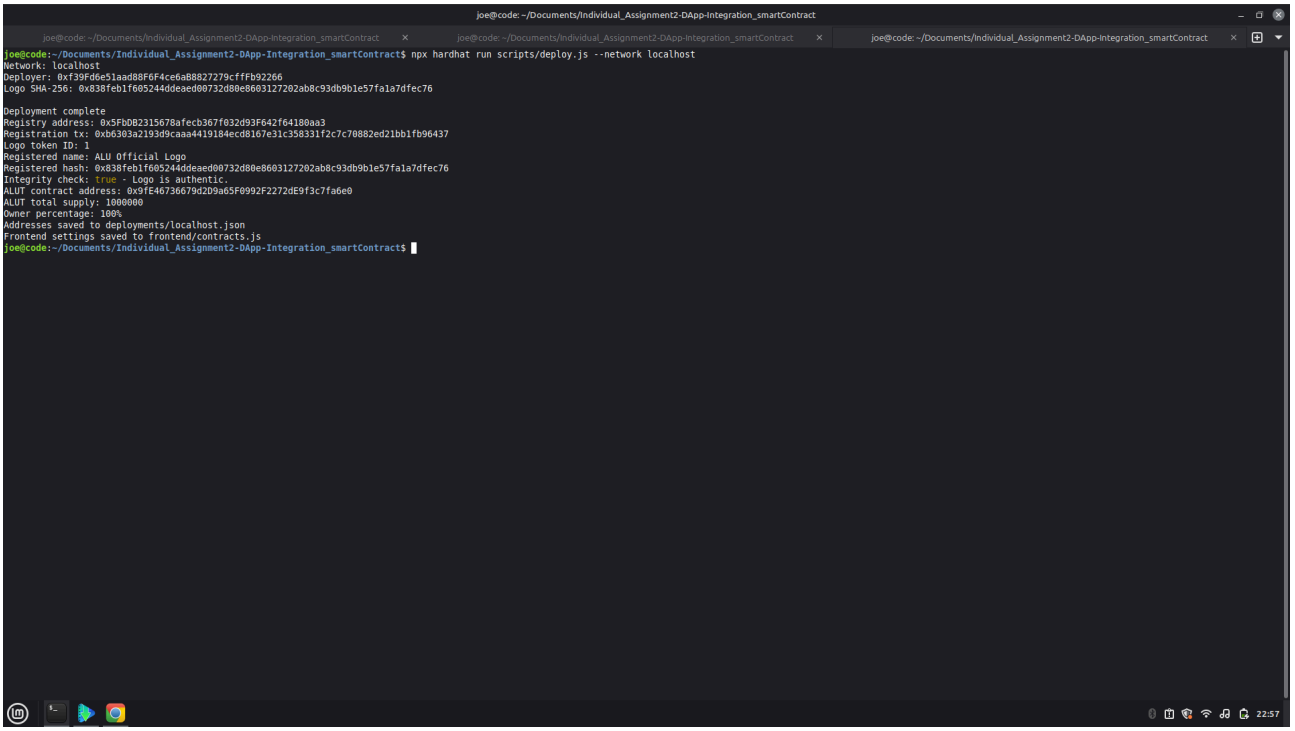


How the web page, the wallet and the two contracts communicate.

The page talks to the blockchain in two ways. To read data, it uses a direct connection to the blockchain node, which is why reading works when no wallet is installed. To change data, it hands the transaction to the user's wallet, which asks the user to approve it, signs it and sends it to the network.

File	Job
frontend/index.html	Page layout: Verify, Register and Token Dashboard sections, with the wallet button in a header shown on every section.
frontend/style.css	Styling, including the green and red verification results.
frontend/logic.js	Hashing, hash validation, the logo check, the supply read and the share transfer. It is kept separate from the page so the automated tests can run the same code.
frontend/app.js	Wallet connection, page navigation, forms and error messages.
frontend/contracts.js	Contract addresses and ABIs. The deployment script writes this file automatically.
scripts/deploy.js	Deploys both contracts, registers the logo and saves the frontend settings.

Running the deployment script on the local blockchain placed both contracts, registered the official logo as token 1, confirmed the integrity check and saved the settings the frontend reads.



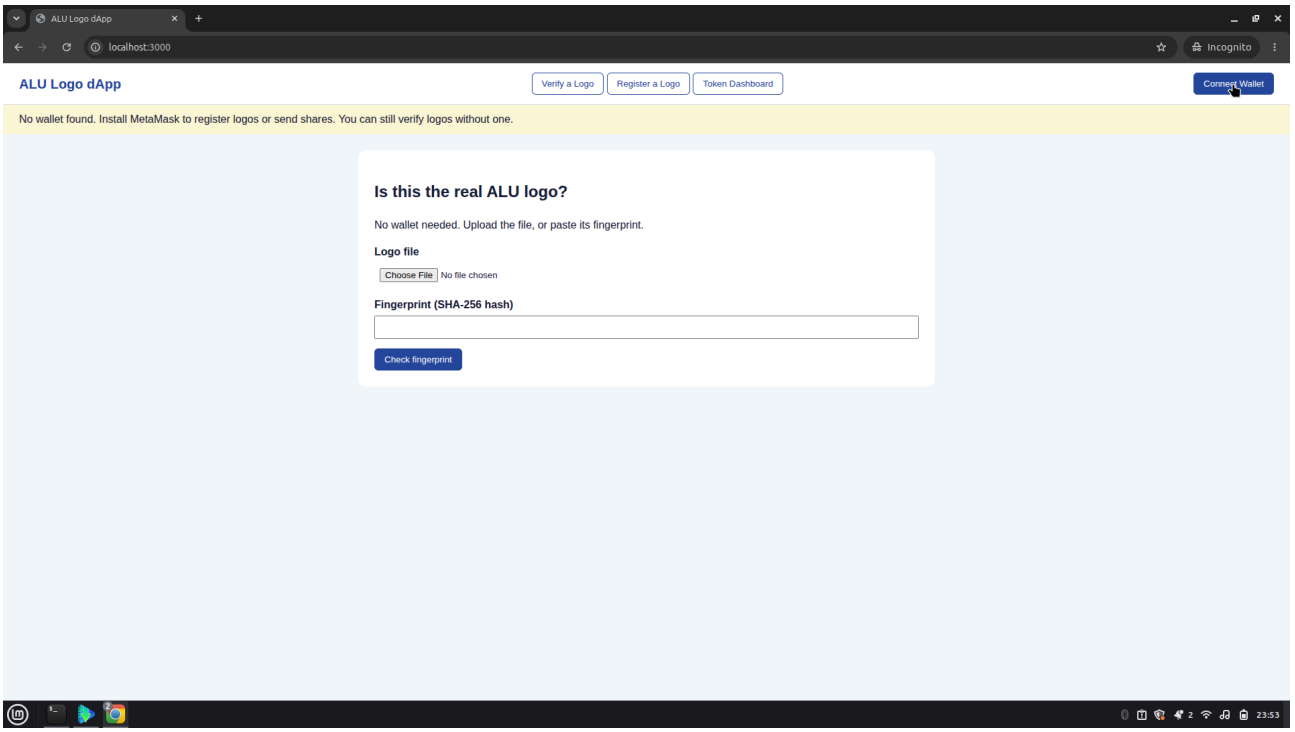
The deployment script: both contracts deployed, the logo registered as token 1, and the frontend settings saved.

Feature Walkthrough

The application has one header and three sections. The header holds the navigation buttons and the Connect Wallet button, so the button is available wherever the user is.

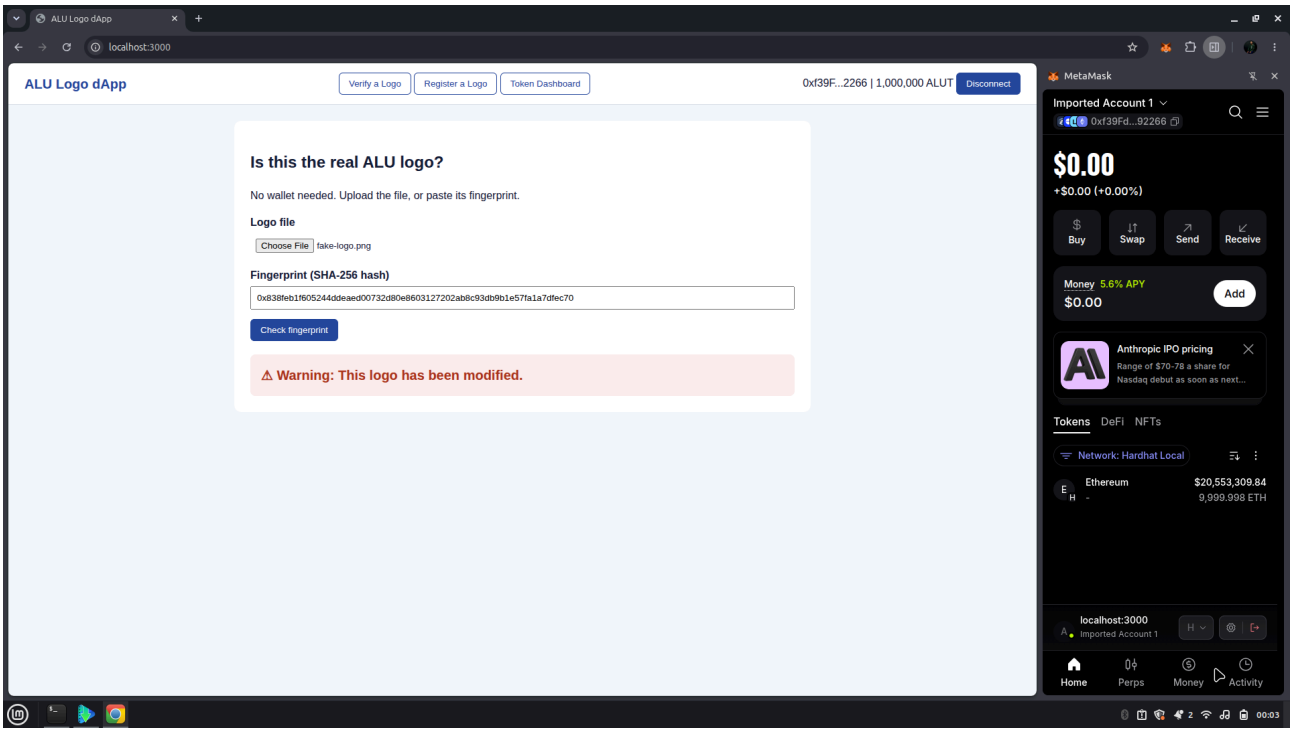
Connecting a wallet

If no wallet is installed, clicking Connect Wallet shows a helpful message instead of failing, and it tells the user that verification still works. I tested this in a private browser window, where the MetaMask extension is not loaded.



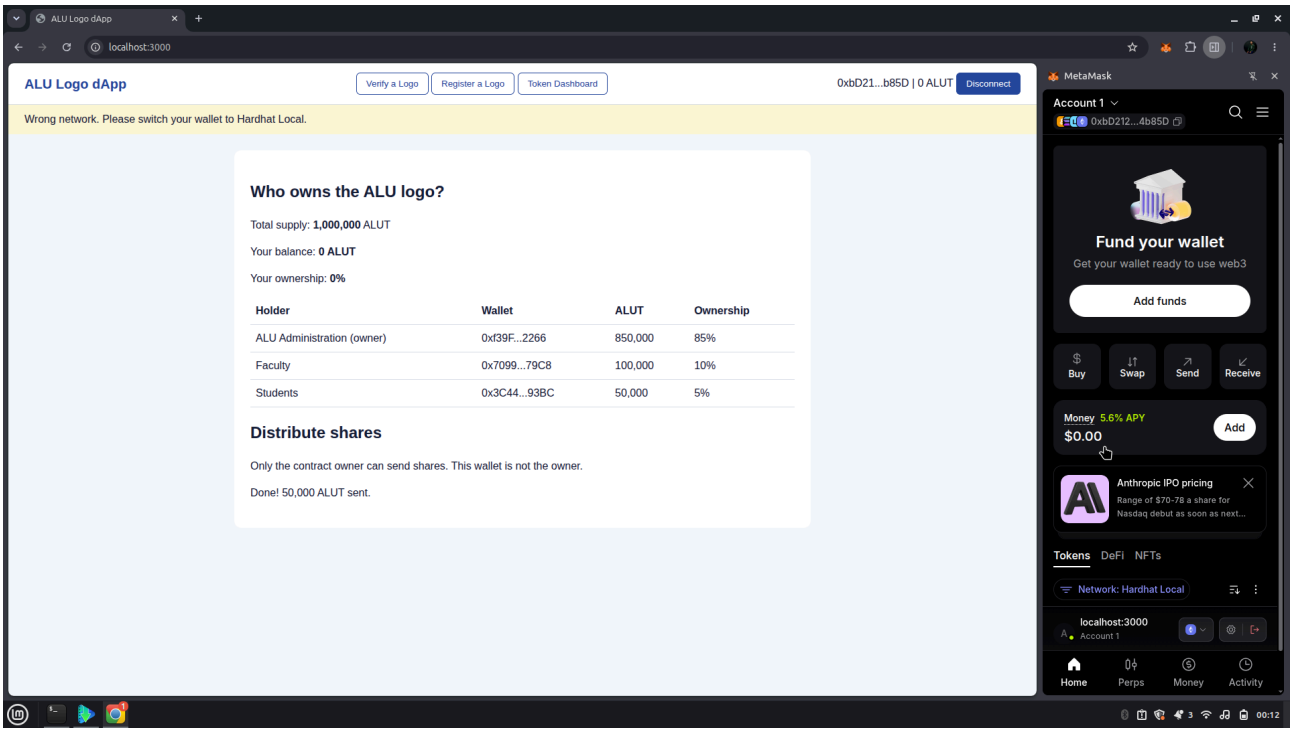
With no wallet installed, the page explains what to do and confirms that verifying still works.

With MetaMask installed, the page asks the wallet to switch to the Hardhat network, adding the network if it is missing, and then requests access to the account. Once connected, the header shows the shortened address and the ALUT balance, and the button changes to Disconnect.

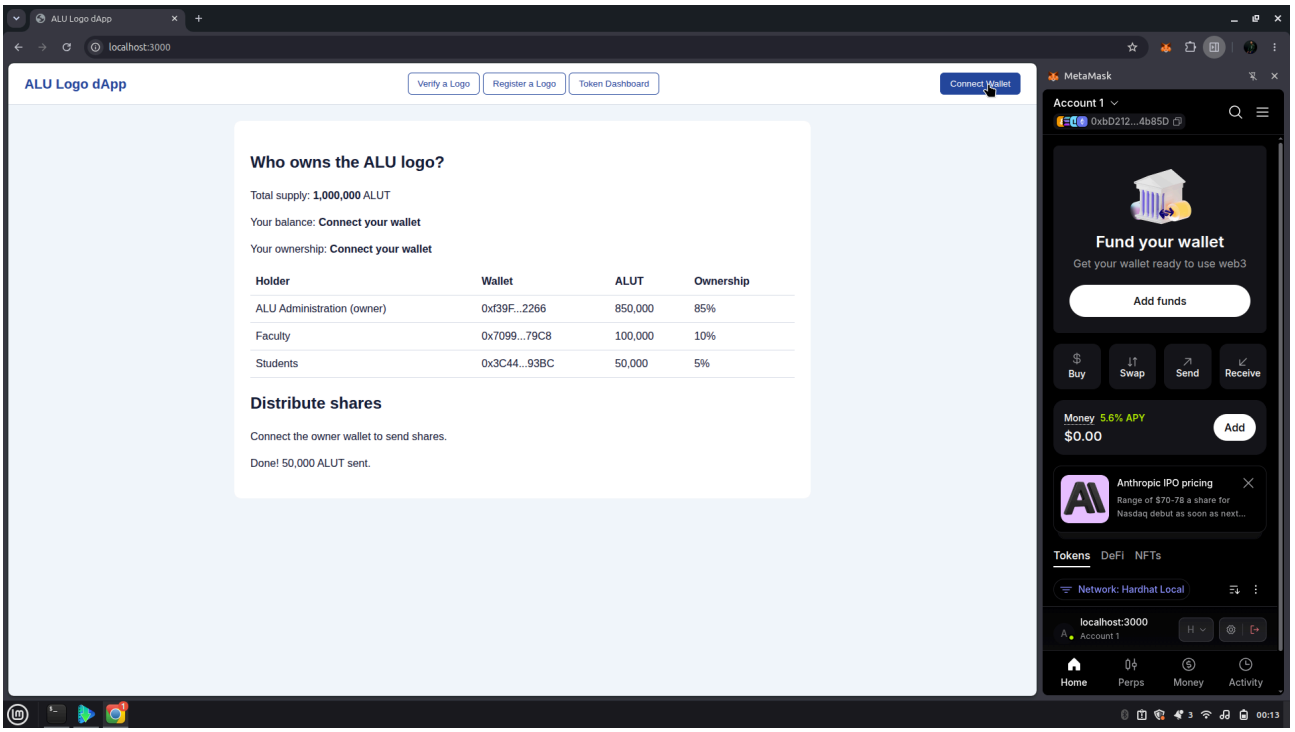


The connected owner wallet: shortened address and ALUT balance in the header.

The page also responds when the user changes something in the wallet. Switching to another account reloads its balance and permissions. Switching to a different network shows a warning. Clicking Disconnect returns the page to its starting state.



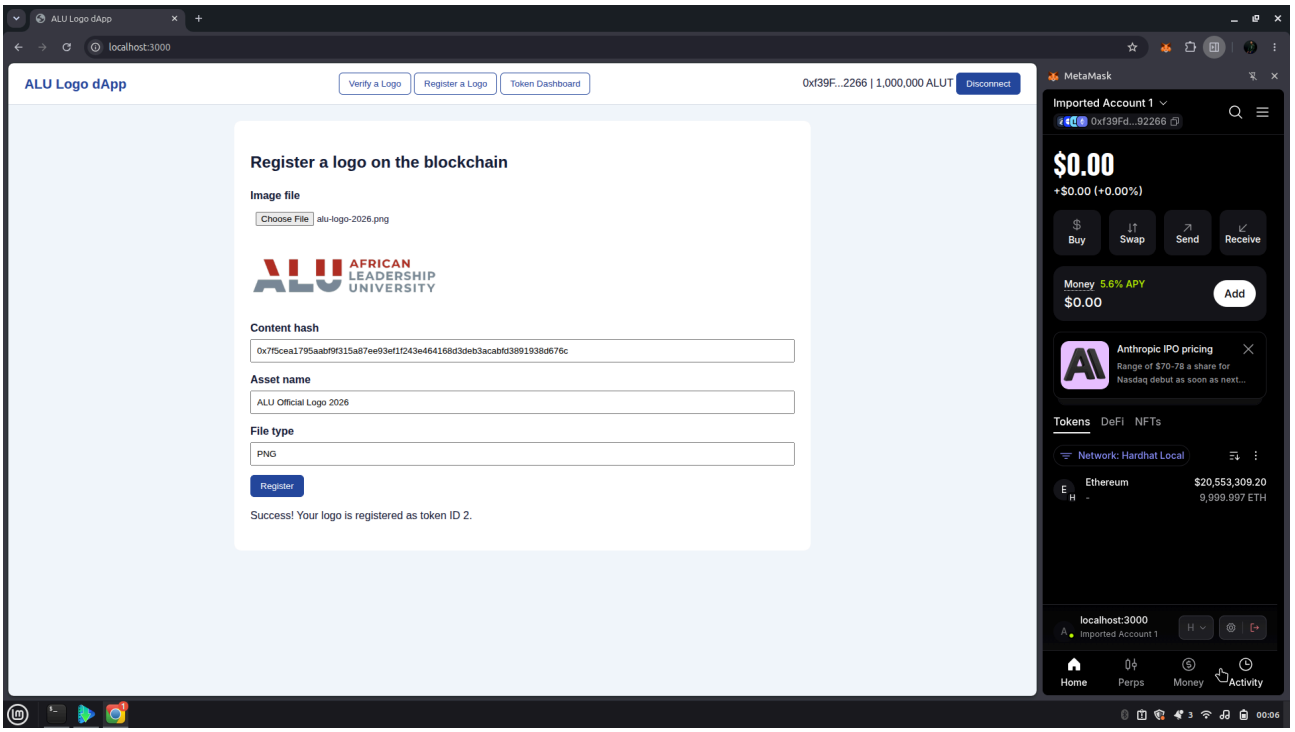
Switching MetaMask to another network shows a clear warning.



After Disconnect, the wallet details are cleared and Connect Wallet returns.

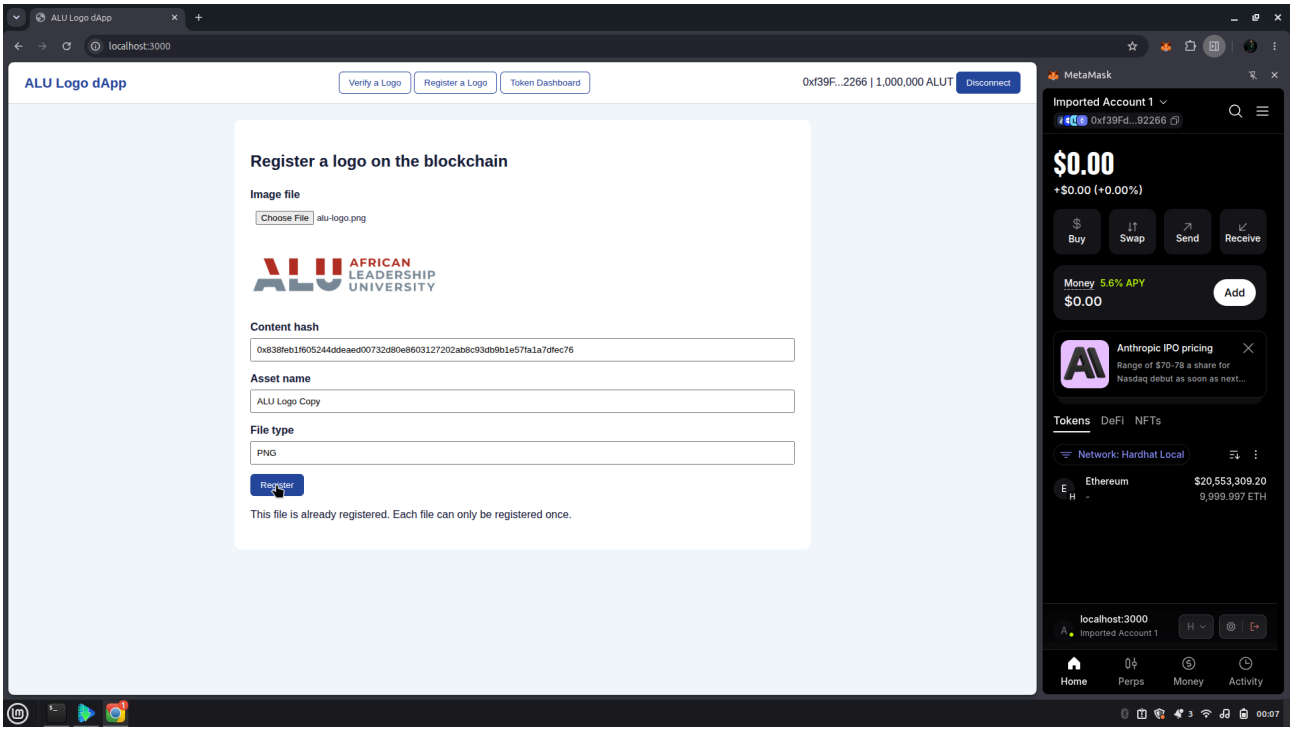
Registering a logo

When a user selects an image, the page shows a preview and computes the file's SHA-256 hash. The hash fills the read-only Content hash field, already in the bytes32 format the contract expects. The user adds an asset name and a file type and clicks Register. MetaMask asks for approval, and when the transaction is confirmed the page reads the new token ID from the contract's AssetRegistered event.



Registering a new logo file: preview, pre-filled hash, and a success message with token ID 2.

Each file can be registered only once. When I tried to register alu-logo.png again under a different name, the page refused it with a clear message.



A file that is already registered is rejected, even under a different name.

Answers on registration

Why is it important to generate the hash in the browser rather than sending the file to a server? The file never leaves the user's computer, so a confidential or unreleased design stays private. There is no server to pay for, secure or keep online, and no server that could be hacked to return a false hash. The user does not have to trust an intermediary, because the calculation happens

on their own device using a standard built into the browser.

What happens at the smart contract level when a user tries to register a hash that already exists? The contract keeps a mapping called registeredHashes. The check *require(!registeredHashes[contentHash], "This file is already registered.")* fails, the transaction reverts, and every change in it is undone, so no token is minted and no record is written. My page checks the same mapping first, for free, so the user normally sees the message before being asked to pay gas. The contract's check is still the real protection, because anyone could call the contract directly without using my page.

What does it mean for the user to approve a transaction in their wallet? They are authorising a specific instruction from their own account: this contract, this function, with this data, and a gas fee paid from their balance. Their private key signs that instruction. Once it is confirmed the action is permanent, and it is publicly recorded that their address made the registration. In this case, their address is stored as the registering wallet and receives the new certificate.

Wallet and Web3 Integration

A wallet such as MetaMask adds an object called window.ethereum to every web page. The page uses it to ask for the user's account. The wallet holds the private key, and the page never sees it.

ethers.js is the library that translates between JavaScript and the blockchain. In this project it does the following work:

- JsonRpcProvider opens the read-only connection to the blockchain node.
- BrowserProvider wraps the wallet, and getSigner returns an object that can request signatures.
- Contract combines an address and an ABI into an object with ordinary JavaScript functions, such as token.totalSupply().
- contract.connect(signer) gives the same contract the ability to send transactions from the user's account.
- transaction.wait() pauses until the transaction is confirmed, and interface.parseLog decodes the event that holds the new token ID.

The frontend reads data from both contracts. The dashboard reads the total supply from ALULogoToken and the verification page reads from ALUAssetRegistry. Both work without a wallet, as the screenshots in this report show.

What is an ABI and why does the frontend need it? ABI stands for Application Binary Interface. Once deployed, a contract is only bytecode, and it receives and returns encoded bytes. The ABI is the list of its functions, parameters, return types, events and errors. ethers.js uses it to encode a call such as verifyLogoIntegrity(1, hash) into the bytes the contract understands, and to decode the answer into values JavaScript can use. Without the ABI, the frontend would know where the contract is but not how to talk to it.

What is the difference between a read-only call and a transaction? A read-only call asks one node to run a view function against its current copy of the data and return the result. Nothing changes, so it needs no gas and no signature, and it returns immediately. A transaction asks the whole network to change the shared state. It must be signed, it costs gas, it has to be included in a block, and its result is permanent. In this project, verifyLogoIntegrity and balanceOf are reads, while registerAsset and distributeShares are transactions.

Why does the user sign transactions with their wallet rather than the application signing for them? For the application to sign, it would need the user's private key, and anyone holding that key

controls the account. If the site were hacked, every user's account would be exposed. Having the wallet sign keeps the key on the user's device and gives them the chance to inspect and refuse each action. It also makes the record meaningful: a signature proves the account holder authorised the action, not the website.

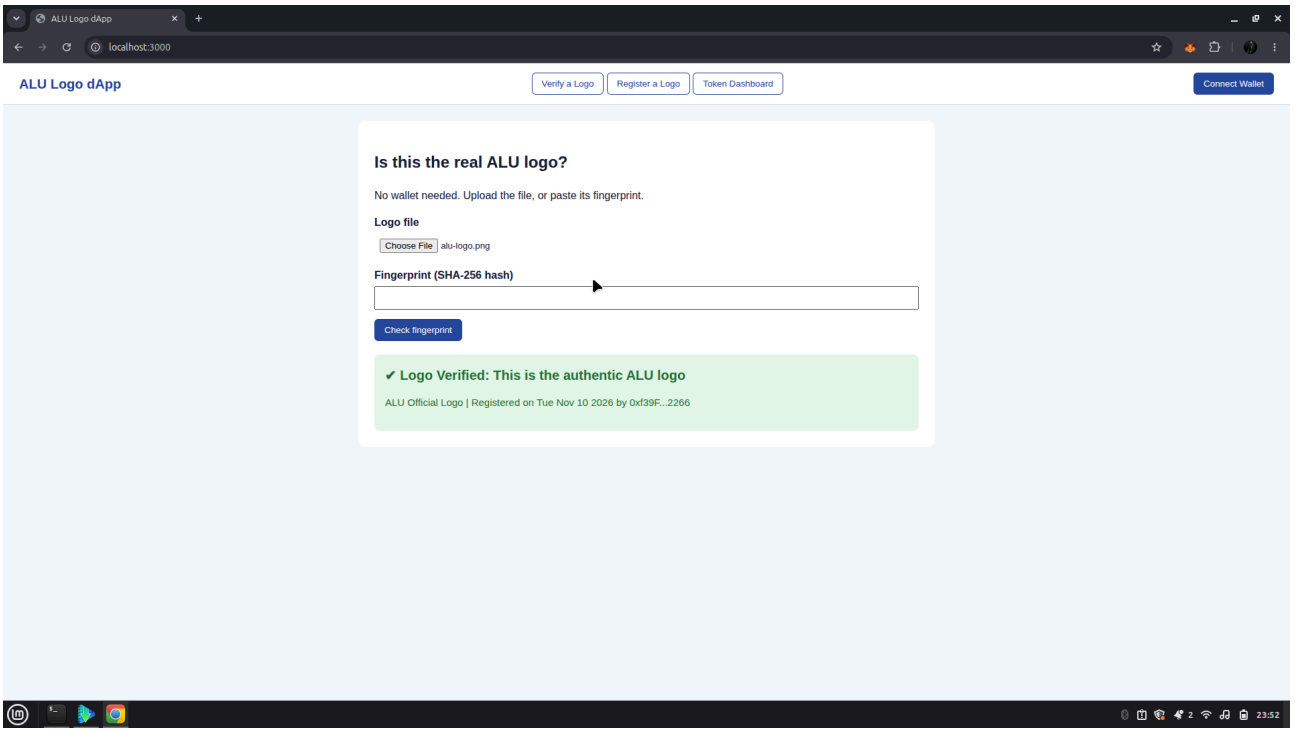
Verification Page

This page is the main public feature. A user uploads any image or pastes a hash, and the page gives a clear answer. It needs no wallet and no account.

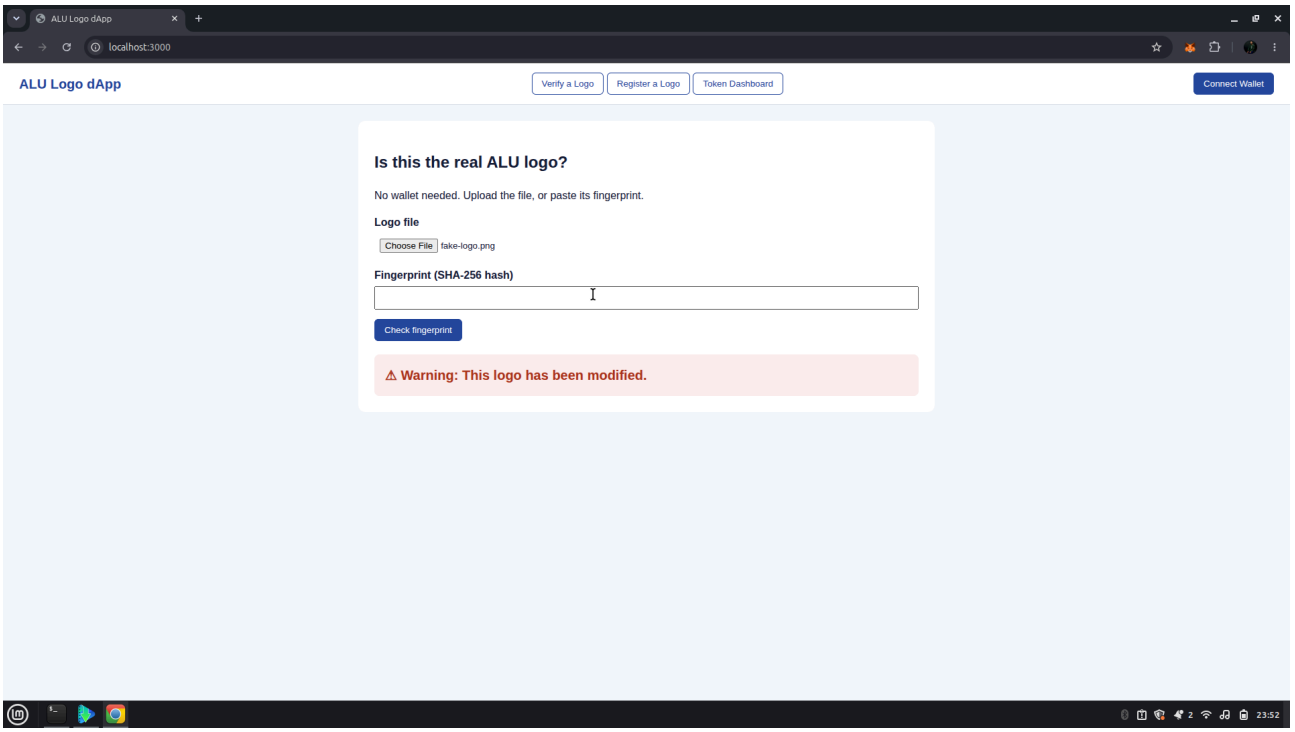
How in-browser hashing works

When a file is selected, the page reads its bytes with file.arrayBuffer(). It passes them to crypto.subtle.digest("SHA-256", bytes), part of the Web Crypto API built into modern browsers. The result is 32 bytes, which the page converts to 64 hexadecimal characters and prefixes with 0x. That is the exact bytes32 format the contract takes. No server is needed because all of the computing happens in the browser and the reference value is already on the blockchain.

The page then calls verifyLogoIntegrity() with the known token ID of the official logo. If the hashes match, it also calls getAsset() and shows the asset name, registration date and registering wallet.

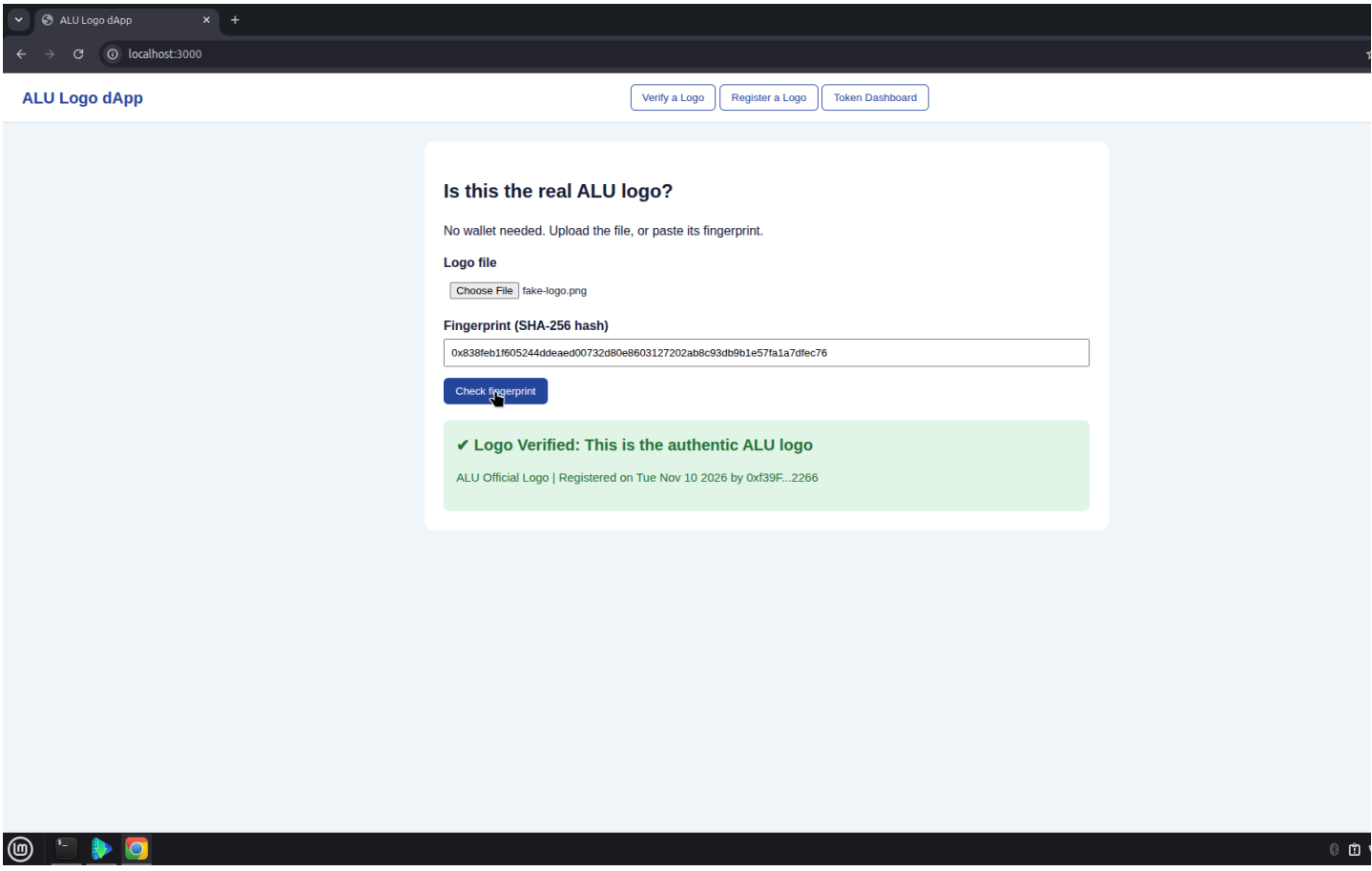


Uploading the real alu-logo.png: a green result with the registered details.

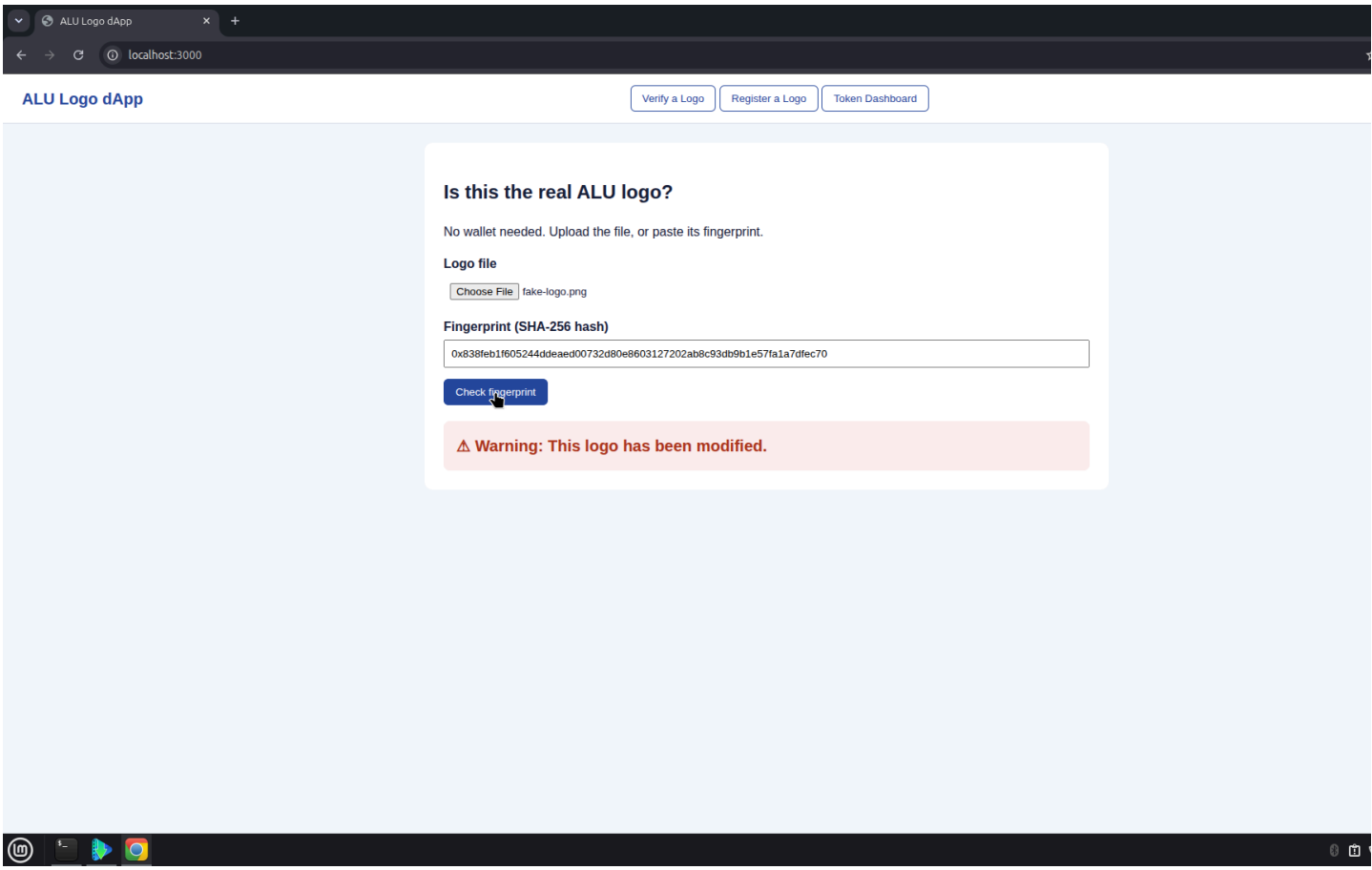


Uploading fake-logo.png: a red warning.

A user who already has a hash can paste it instead. The page accepts it with or without the 0x prefix and rejects text that is not a valid hash. Changing only the last character of the correct hash is enough to produce the warning.



Pasting the correct hash: verified.



Pasting a hash with its last character changed: the warning appears.

The result is designed for a non-technical reader. It uses colour, a tick or warning symbol and a plain sentence, and the only address shown is the shortened registering wallet that the assignment asks for. The page calls the hash a fingerprint, and it shows no raw hexadecimal in the result.

Answers on verification

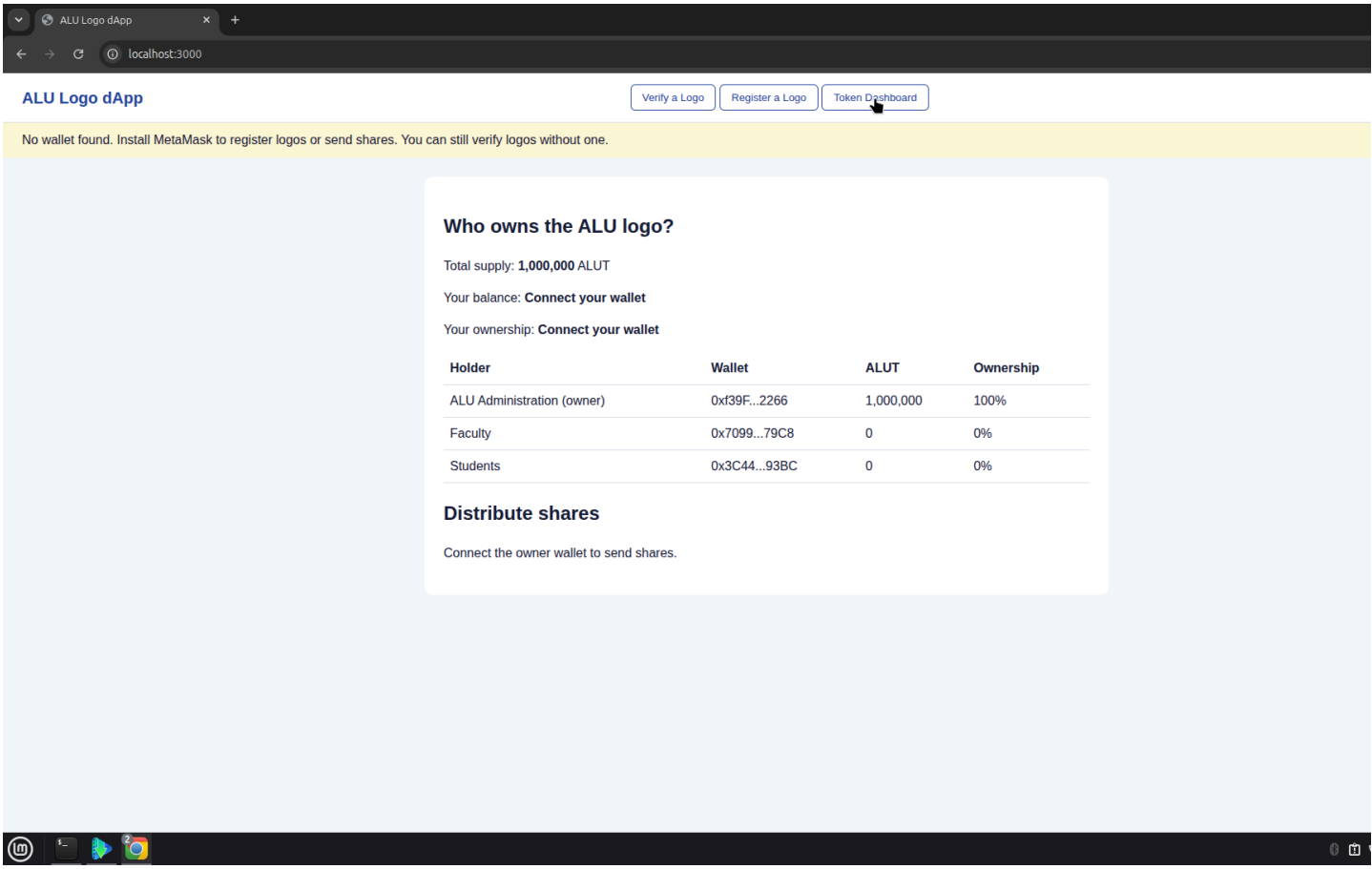
Why does this page not require a wallet, what type of call is verifyLogoIntegrity(), and why does it cost no gas? A wallet is needed only to sign, and this page signs nothing. verifyLogoIntegrity() is declared as a view function, meaning it can read the contract's storage but cannot change it. A node runs it locally and returns the answer. Gas pays for network-wide computation and permanent storage, and a view call made this way uses neither.

Give a real-world scenario in which someone would use this page. An employer in Kigali receives a certificate from a job applicant, along with a logo file the applicant says came from ALU. The HR officer has no blockchain knowledge and would otherwise have to email the registrar and wait. On this page they upload the file and learn within seconds whether it matches the registered logo. The same page helps a merchandise supplier confirm that the artwork they were sent is the approved version.

Why would someone checking a downloaded logo trust the result? The answer comes from a public record that ALU cannot quietly edit, because the contract has no function to change or delete a registration. The hash is calculated on the checker's own device, so no server can alter that step. SHA-256 makes it practically impossible to create a different file with the same fingerprint. A cautious user does not have to trust my page either: on a public network they can read the same record on Etherscan, as Formative 1 showed. The check proves that a file matches the registered record. It relies on the user being directed to ALU's genuine contract address, which ALU would need to publish on its official channels.

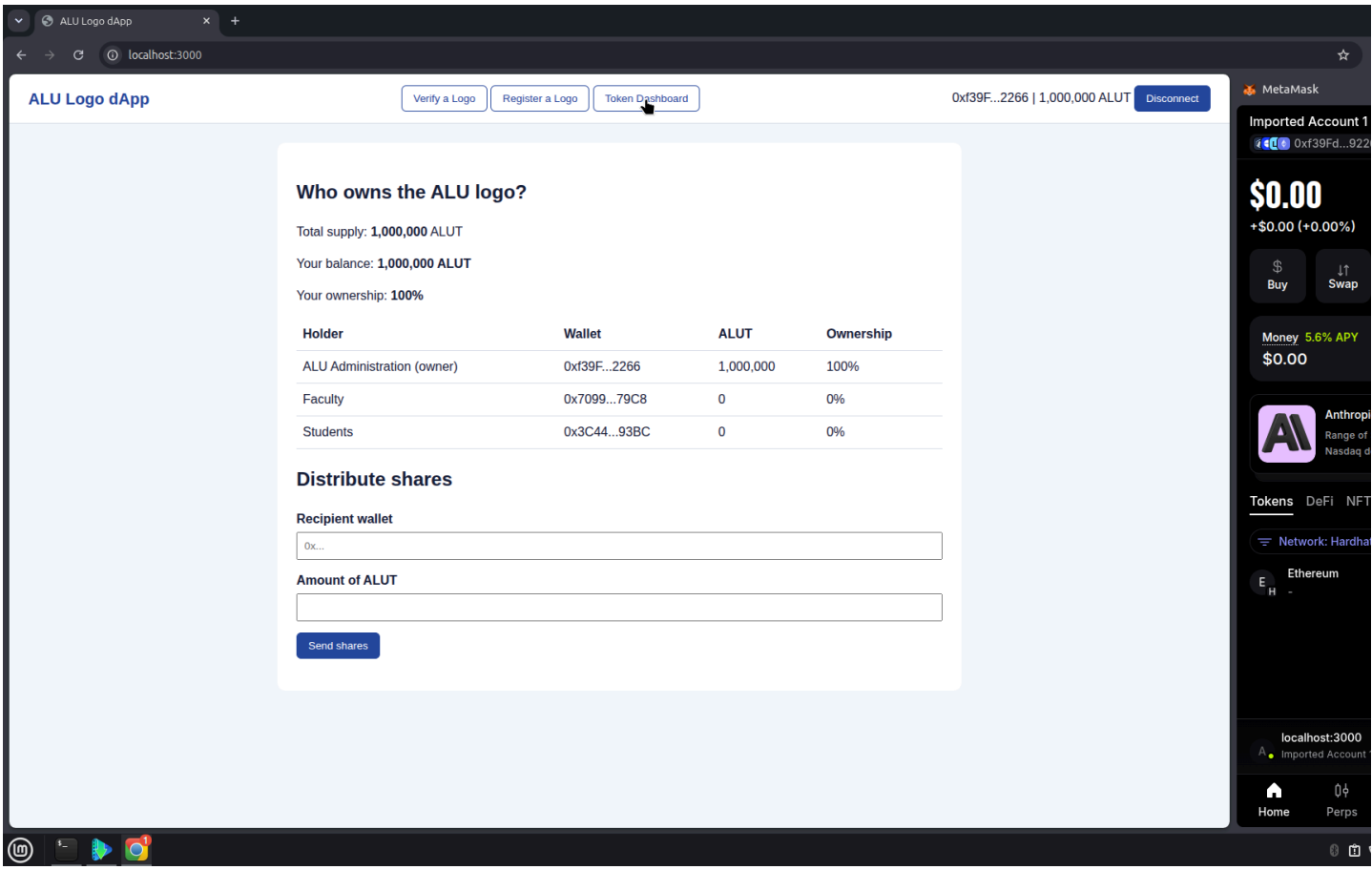
Token Dashboard

The dashboard shows who owns the ALU logo. All figures come directly from the ALULogoToken contract: the total supply of 1,000,000 ALUT, the connected wallet's balance, its percentage from ownershipPercentage(), and a table of three example holders. Anyone can see the supply and the table without connecting a wallet.



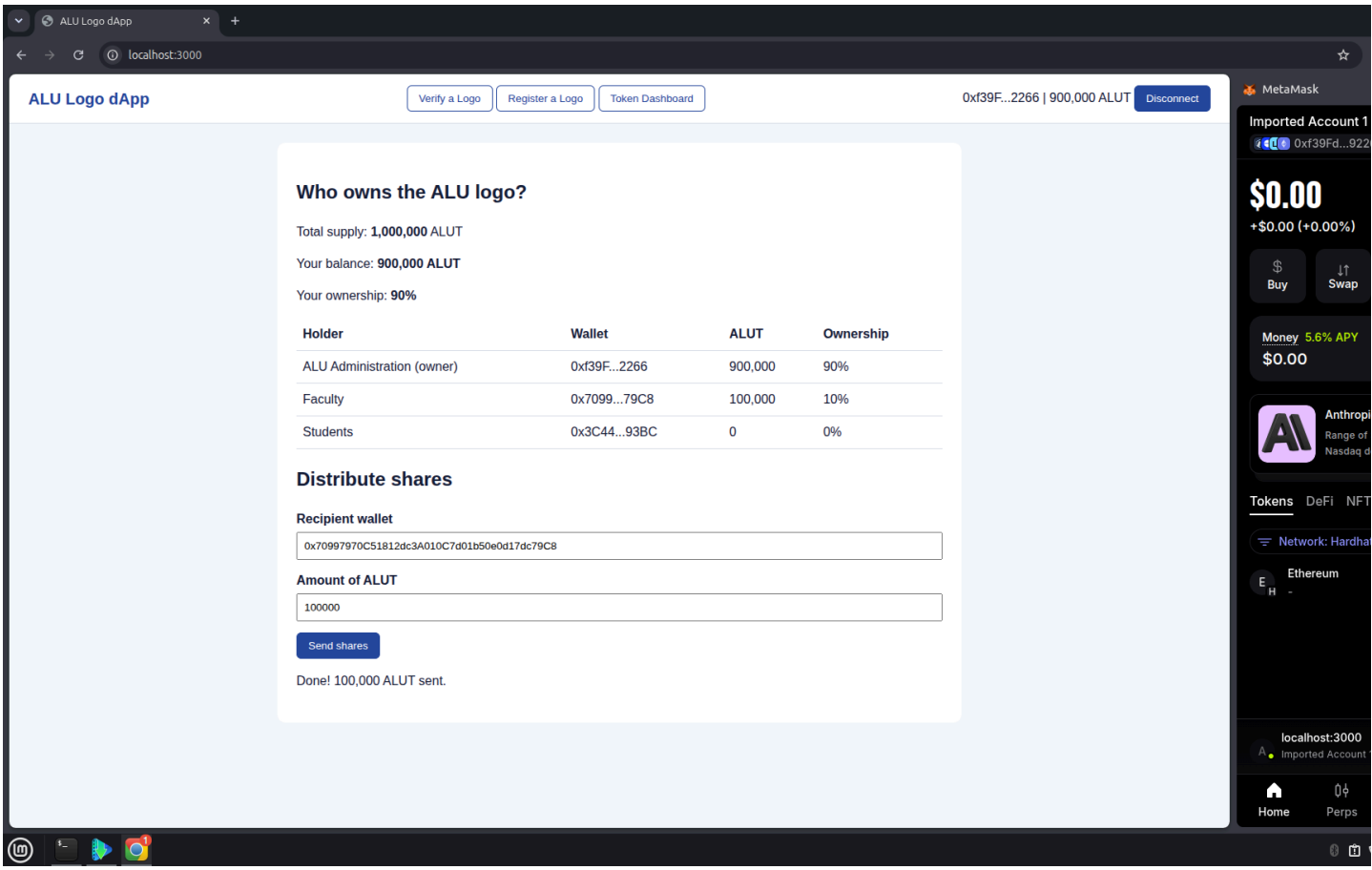
The dashboard without a wallet: public figures are still visible, and the distribution form is hidden.

When the contract owner connects, the Distribute shares form appears. The page compares the connected address with the contract's owner() value to decide this.

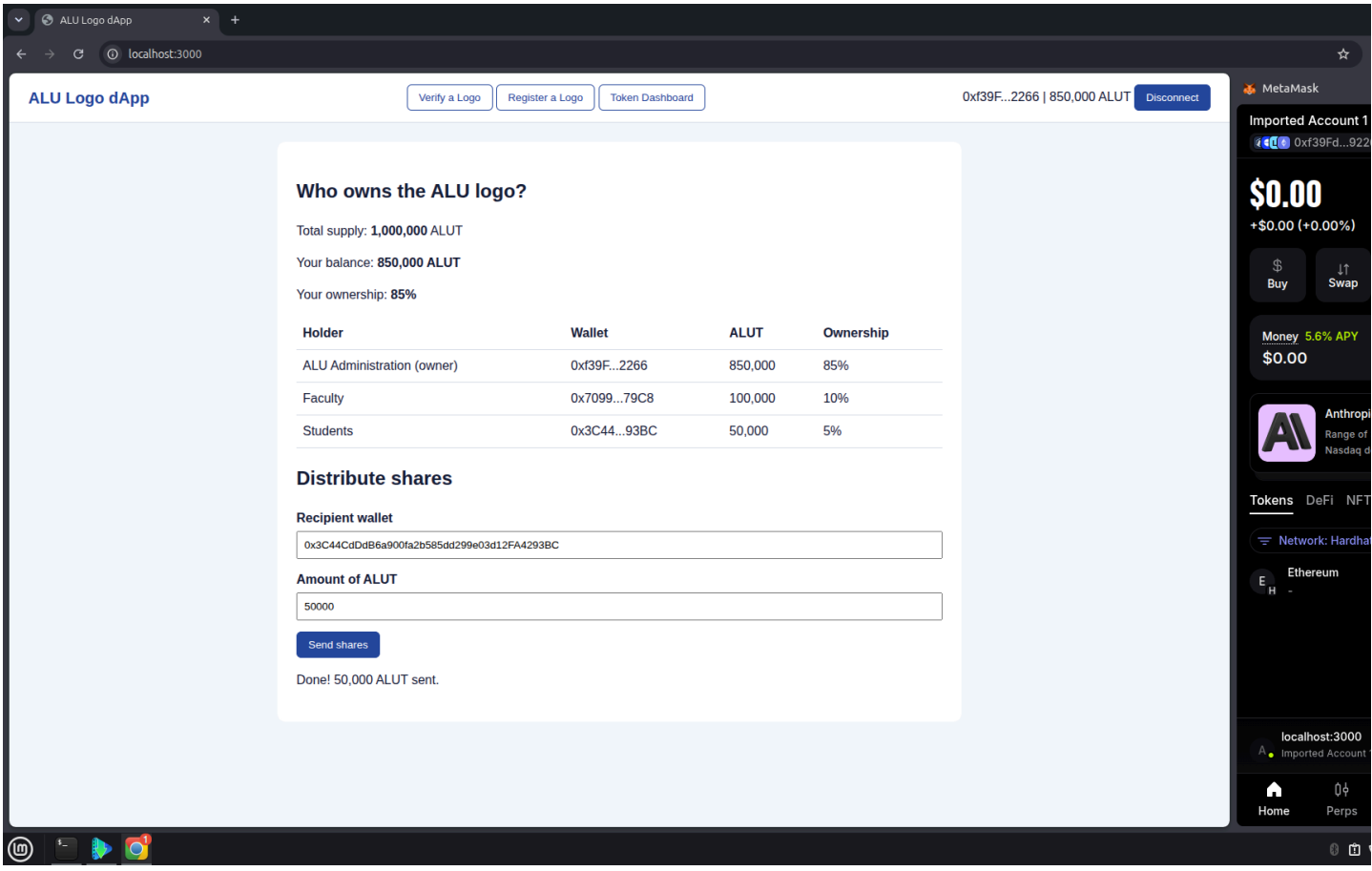


The owner's view before any distribution: 100% held by ALU, with the form visible.

The form checks that the recipient is a valid address, that the amount is greater than zero and that the owner has enough ALUT. It then calls distributeShares() and MetaMask asks for approval. After confirmation, the page reloads the balances from the contract. I sent 100,000 ALUT to the Faculty wallet and then 50,000 ALUT to the Students wallet.

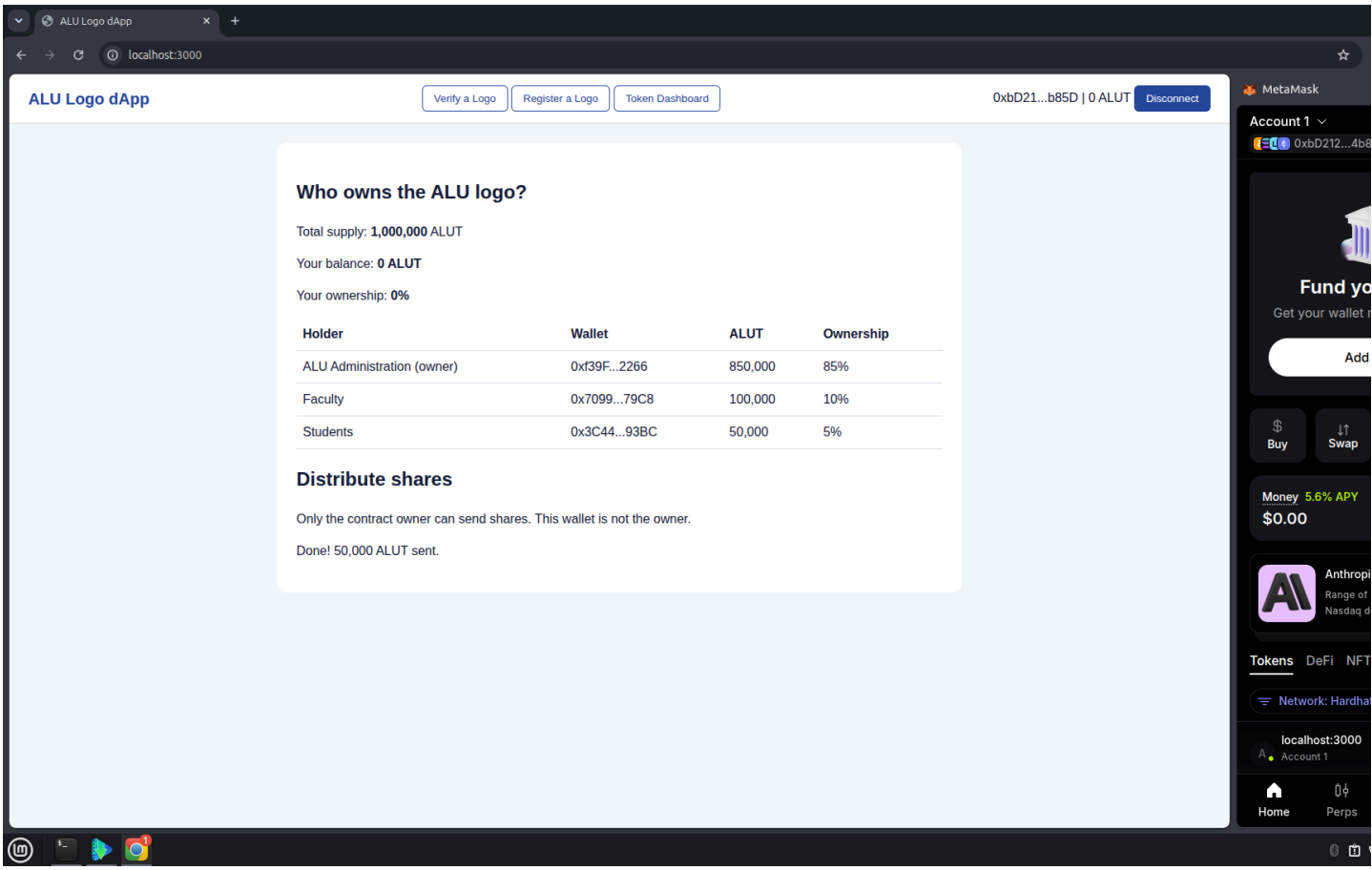


After sending 100,000 ALUT to Faculty: 90% and 10%.



After sending 50,000 ALUT to Students: 85%, 10% and 5%.

A wallet that is not the owner does not see the form and is told why.



A non-owner wallet holding 0 ALUT: the form is hidden and a message explains that only the owner can send shares.

How distributeShares() is protected

Hiding the form is only a convenience, because anyone can call a public contract without using my website. The real protection is in the contract, where distributeShares() carries the onlyOwner modifier from OpenZeppelin's Ownable. The modifier runs before the function body and compares the caller with the stored owner. If a non-owner calls it, the transaction reverts with the OwnableUnauthorizedAccount error, no tokens move, and the caller still pays for the gas used up to that point. Automated test 7 confirms this.

Answers on distribution

Who at ALU might receive ALUT, and what would holding it mean in practice? A long-serving faculty member who helped build ALU's academic reputation might receive 10,000 ALUT, which is 1% of the shares. In practice, the holding is public proof of a recognised stake in the logo. If ALU earned licensing income from official merchandise and agreed a payout policy, that person could receive 1% of it. If decisions about the logo were put to a vote, they could hold 1% of the voting power. These rights exist only if ALU sets them out in a real agreement, because the token records the share but does not create legal rights by itself.

What would need to change for shareholders to vote in proportion to their holdings? ALULogoToken would need OpenZeppelin's ERC20Votes extension, which records each holder's balance at past blocks and lets holders delegate their votes. Without those snapshots, someone could vote, transfer the tokens to another wallet and vote again. A governance contract, such as

OpenZeppelin's Governor, would then handle proposals, the voting period, the quorum and counting. Sensitive actions would need to move from the single owner to that governance contract. Because the current contract cannot be upgraded, this would require deploying a new token and moving the existing balances across to it.

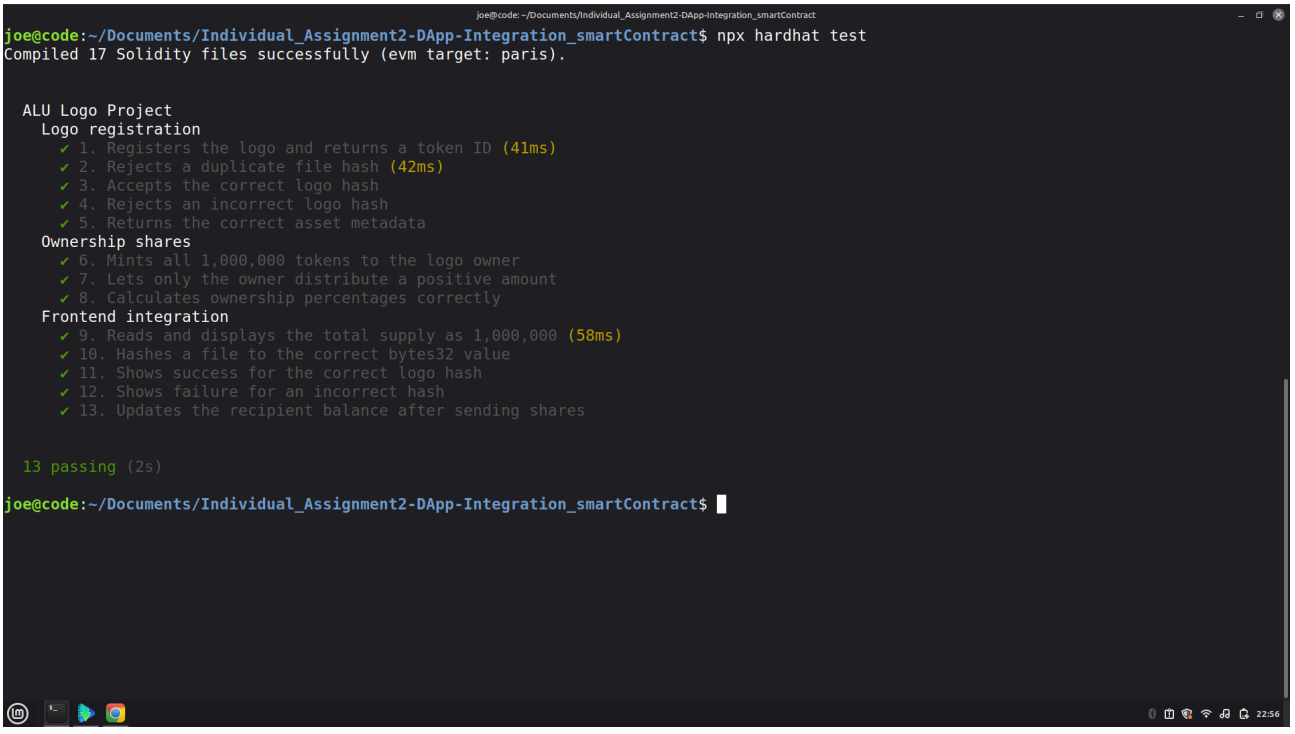
Test Results

Automated tests are small programs that check the main programs behave as intended. Each of the thirteen tests starts from a fresh blockchain, so they cannot affect each other, and the logo tests use the real logo's fingerprint.

#	**What the test checks**	**Why it matters**
1	The ALU logo registers and receives certificate number 1.	Registration works.
2	Registering the same logo a second time is refused.	Nobody can register the logo twice or claim it after ALU.
3	Checking the real logo's fingerprint answers "Logo is authentic."	Real files are recognised.
4	Checking a fingerprint that differs by the smallest amount answers "Warning: logo does not match."	Fakes are detected, even tiny changes.
5	Looking up the record returns the right name, file type, fingerprint, owner and time.	Records can be looked up correctly.
6	All 1,000,000 shares are created and given to ALU, with the right name and symbol.	Ownership starts at 100% with ALU.
7	Handing out 100,000 shares works, while anyone other than ALU, or an amount of zero, is refused.	Only ALU can hand out shares.
8	An account holding 500,000 shares is reported as owning 50%.	Percentages are calculated correctly.

Five further tests check the frontend integration. They import the functions in frontend/logic.js, so they exercise the same code the web page runs, against freshly deployed contracts.

#	What the test checks	Why it matters
9	The frontend reads the total supply and formats it as 1,000,000.	The dashboard shows the right figure.
10	A file passed to the hashing function returns the correct SHA-256 hash in bytes32 format.	Browser hashes match what the contract stores.
11	verifyLogoIntegrity() with the correct hash produces the success result.	Real logos are confirmed.
12	verifyLogoIntegrity() with an incorrect hash produces the failure result.	Modified logos are flagged.
13	distributeShares() updates the recipient's balance.	Share transfers work through the frontend code.



All thirteen tests passing.

Conclusion

What I learned. A fingerprint is a cheap and powerful way to protect a file: the blockchain stores a tiny code, not the image, yet it catches any change to the image. Choosing the right kind of token matters: a unique certificate for one item, and interchangeable shares for ownership. Reusing trusted building blocks, saving records before handing anything over, and locking sensitive actions to the owner prevent serious security mistakes. Because blockchain mistakes

are permanent, practising locally, testing automatically and trying a public test network first are essential. Building the web application taught me how an ABI and ethers.js connect JavaScript to a contract, why reading and writing take different paths, and why the private key must stay in the user's wallet. It also showed me that frontend checks are for the user's convenience, and that security has to be enforced in the contract.

What I would add or improve with more time.

- Limit who can register. Today any account can register any file; in real use only ALU-approved accounts should, ideally with approval from several ALU officials.
- Keep the logo image itself on a shared online file network and link it to the certificate, so the original can always be downloaded, not only checked.
- Link the shares to the certificate, so the shares program records exactly which registered logo it represents.
- Show more precise percentages, since holdings below 1% currently show as 0.
- Point the web application at Sepolia and host it online, since it currently runs against the local Hardhat network, so anyone can use it from a public link.
- Add voting or income sharing so that share holders can actually use their shares.

GitHub Link and Video Demonstration

Item	**Link**
GitHub repository	https://github.com/josep-prog/Individual_Assignment2-DApp-Integration_smartContract
Demo video	https://youtu.be/fiC_qeBMTC0
ALUAssetRegistry on Etherscan	sepolia.etherscan.io/address/0xA471fB76f0c2432Faf36d5f699C8868FAF0ab3B9
ALULogoToken on Etherscan	sepolia.etherscan.io/address/0x9C0B96aA3A3b564a03fad5b53d23775d066d44C3

Term	**Meaning**
Blockchain	A shared record book copied across thousands of computers, where new pages (blocks) are added but old pages cannot be changed.
Ethereum	A blockchain that can run programs (smart contracts), not only record payments.
Smart contract	A program stored on the blockchain that runs exactly as written and cannot be secretly edited.
Solidity	The programming language of Ethereum smart contracts.
Deploy	Publishing a contract onto a blockchain, which gives it an address.

Wallet address	A public ID on the blockchain, like an account number.
Secret key	The secret that controls an address. Whoever has it controls the funds.
Transaction hash	The unique 64-character receipt number of a transaction.
Hash (SHA-256)	A fixed-length fingerprint of data; any change to the data gives a completely different fingerprint.
Token	A digital unit managed by a smart contract.
NFT, ERC-721	A unique, non-interchangeable token, and the standard that defines it.
Fungible, ERC-20	Interchangeable tokens, like money, and the standard that defines them.
OpenZeppelin	A trusted, audited library of standard smart-contract code.
Hardhat	A developer toolkit to compile, test and deploy contracts, including a local blockchain.
Testnet (Sepolia)	A public practice version of Ethereum whose ETH has no value.
Etherscan	A public website for looking up any address, transaction or contract.
Tokenisation	Splitting ownership of an asset into digital tokens.